StarAgile

PROJECT IMPLEMENTATION REPORT

Zomato DevOps CI/CD Platform on AWS

Author: Jeni Balar
Role: Cloud / DevOps Infrastructure Project
Repository: View Project On Github

Core Cloud: AWS (VPC, EC2, IAM, NAT Gateway, Security Groups, Internet Gateway)
Application: Zomato Web Application
Infrastructure: GitHub | Jenkins | Docker | Amazon ECR | Amazon EKS | Kubernetes | Helm
Monitoring: Prometheus | Grafana | Alertmanager | Node Exporter | Kube State Metrics | Prometheus Operator | metrics-server

[EXECUTIVE ABSTRACT]

This technical document outlines the implementation of an AWS-based Zomato DevOps CI/CD platform using GitHub, Jenkins, Docker, Amazon ECR, Amazon EKS, Kubernetes, Helm, Prometheus and Grafana. The project demonstrates an end-to-end workflow covering source control, automated CI/CD, container image management, Kubernetes deployment, application access and infrastructure monitoring.

Key achievements include:

- Automated CI/CD workflow from GitHub source code through Jenkins to Kubernetes deployment.
- Docker image creation and private Amazon ECR storage.
- Amazon EKS deployment using Kubernetes Deployment and NodePort Service resources.
- Two-Availability-Zone VPC design with two public and two private subnets.
- Prometheus Community monitoring stack with Prometheus, Grafana, Alertmanager, Node Exporter, Kube State Metrics and Prometheus Operator.
- Grafana dashboard with six monitoring panels for CPU usage, memory usage, node count, pod count, pod CPU usage and pod memory usage.
- End-to-end verification of the Zomato application through Kubernetes port forwarding and browser access.

INDEX

1. Project Overview

The Zomato DevOps project demonstrates an end-to-end cloud deployment workflow for a web application using AWS, GitHub, Jenkins, Docker, Amazon ECR, Amazon EKS and Kubernetes. The source code is maintained in GitHub, while Jenkins provides the CI/CD automation used to build the application, create the Docker image, publish the image to Amazon ECR and deploy the application to Amazon EKS.

The AWS infrastructure is built inside a dedicated VPC with CIDR 10.0.0.0/16 and four subnets distributed across two Availability Zones. Jenkins runs on an EC2 instance in a public subnet, while EKS worker infrastructure and application workloads operate in the private subnets. The Kubernetes application is exposed through a NodePort service on port 31403; for final browser verification, the service is port-forwarded from port 80 to port 8081 on the Jenkins EC2 host.

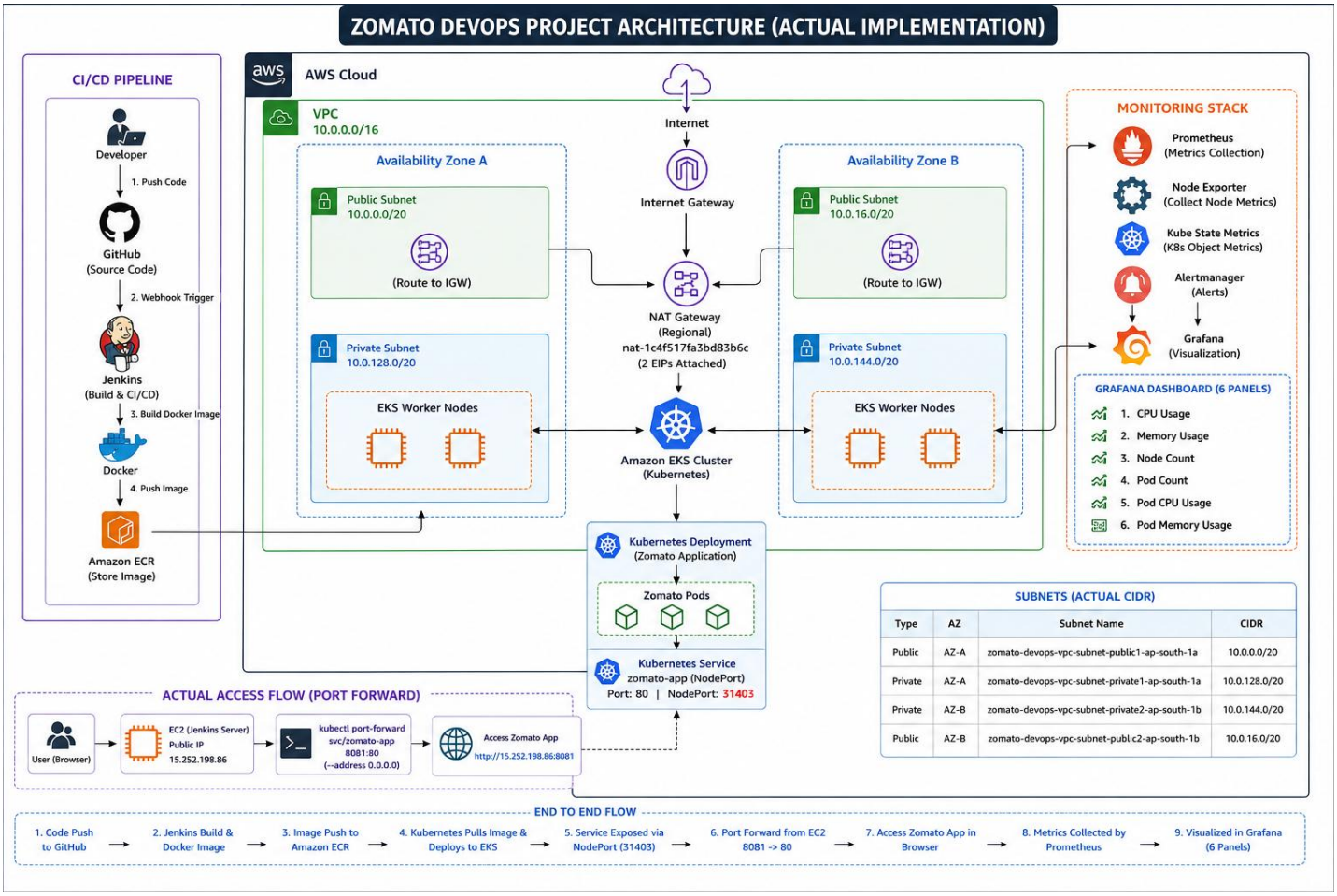
The project also implements Kubernetes monitoring using the Prometheus Community monitoring stack. Prometheus collects Kubernetes and workload metrics, while Grafana visualizes the collected metrics. The monitoring environment includes Prometheus, Grafana, Alertmanager, Node Exporter, Kube State Metrics, Prometheus Operator and metrics-server.

The completed implementation covers source control, CI/CD automation, containerization, private image storage, Kubernetes deployment, application access and centralized monitoring.

2. Objectives

- Automate the application delivery lifecycle from GitHub source code to Amazon EKS using Jenkins.
- Containerize the Zomato application using Docker and store the resulting image in Amazon ECR.
- Deploy the containerized application on Amazon EKS using Kubernetes Deployment and Service resources.
- Use AWS VPC, public/private subnets, Internet Gateway and NAT Gateway to provide the required network foundation.
- Run the Jenkins CI/CD server on an EC2 instance with IAM-based AWS access.
- Configure and verify Amazon EKS Auto Mode and Ready Kubernetes nodes.
- Expose the Zomato application through a Kubernetes NodePort service using NodePort 31403.
- Implement Kubernetes observability using Prometheus and Grafana.
- Deploy and verify Alertmanager, Node Exporter, Kube State Metrics, Prometheus Operator and metrics-server.
- Create a Grafana dashboard containing six monitoring panels for CPU usage, memory usage, node count, pod count, pod CPU usage and pod memory usage.
- Verify the complete deployment through Kubernetes status checks, port forwarding and browser access.

3. Architecture



3.1 Architecture Explanation

- The Zomato DevOps project uses a cloud-native AWS architecture in which the application is containerized with Docker and deployed on Amazon EKS using Kubernetes. The infrastructure is deployed inside a dedicated VPC with CIDR 10.0.0.0/16 and spans two Availability Zones: AZ-A (ap-south-1a) and AZ-B (ap-south-1b).
- The VPC contains four subnets distributed across the two Availability Zones. The actual subnet CIDRs used in the implementation are:

Public AZ-A: 10.0.0.0/20
Private AZ-A: 10.0.128.0/20
Private AZ-B: 10.0.144.0/20
Public AZ-B: 10.0.16.0/20

3.2 End-to-End Flow

- Developer pushes application source code to GitHub.
- GitHub repository provides the source for the Jenkins CI/CD pipeline.
- Jenkins checks out the code, builds the Docker image and pushes it to Amazon ECR.
- Amazon EKS pulls the container image from ECR and deploys the Zomato workload through Kubernetes.
- The Kubernetes Deployment maintains two application replicas and the NodePort Service exposes service port 80 through NodePort 31403.
- For final external verification in the actual implementation, the zomato-app service is port-forwarded from port 80 to port 8081 on the Jenkins EC2 host.
- The user accesses the Zomato application through the EC2 public access path.
- Prometheus collects Kubernetes metrics and Grafana visualizes them in the monitoring dashboard.

3.3 Network and Security Flow

- Jenkins is hosted on an EC2 instance in the public subnet and uses an IAM role for AWS access.
- The EKS worker infrastructure and Zomato application workloads run in the private subnets.
- Private resources use the NAT Gateway for required outbound Internet connectivity.
- The Internet Gateway provides Internet connectivity for the public subnet networking path.
- Kubernetes service access to the Zomato application is implemented with NodePort 31403, while final browser verification uses port forwarding 8081 → service port 80.
- Application and monitoring components are kept inside the Kubernetes cluster and monitoring namespace rather than being directly exposed as independent public services.

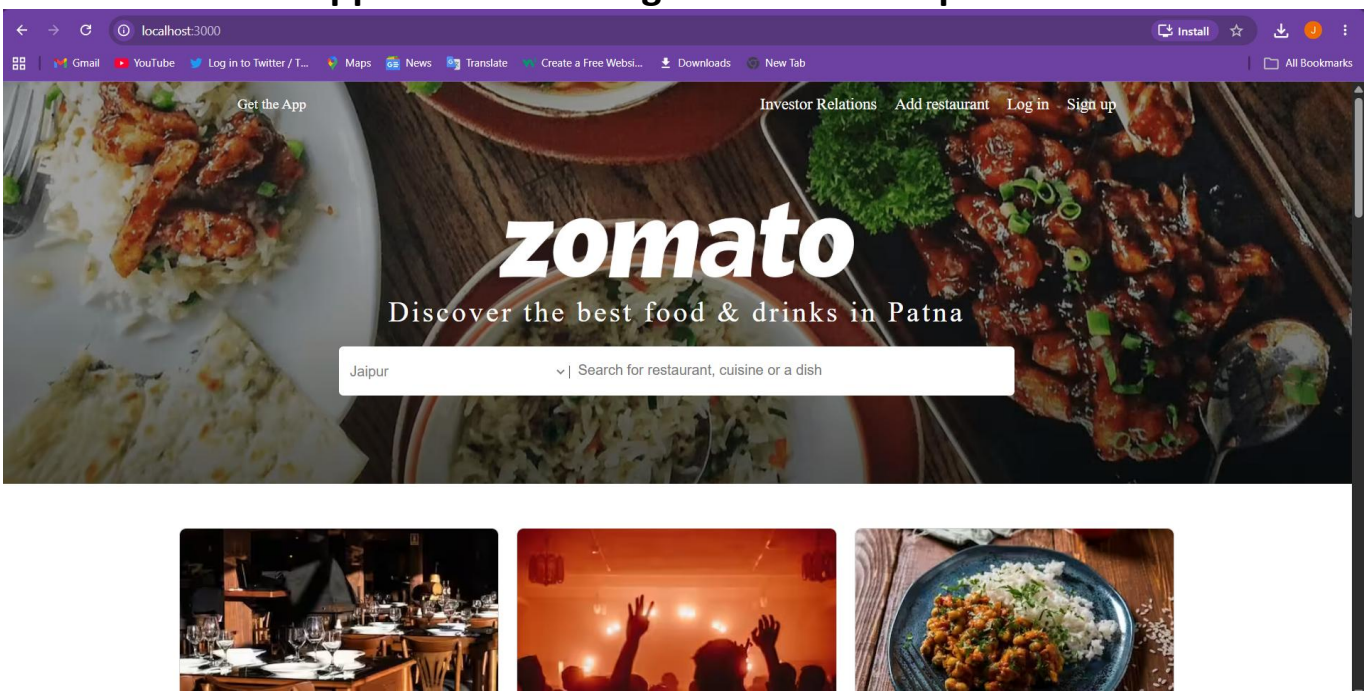
3.4 AWS Services / Technologies Used

Service / Component	Purpose
Amazon VPC	Provides the isolated network environment
Public & Private Subnets	Separates public infrastructure from private Kubernetes workloads.
Internet Gateway	Provides Internet connectivity for public networking.
NAT Gateway	Provides outbound Internet access for private resources.
Amazon EC2	Hosts the Jenkins CI/CD server.
Security Groups	Controls network traffic to AWS resources.
GitHub	Hosts the Zomato application source code and deployment files.
Jenkins	Automates the CI/CD pipeline.
Docker	Builds the Zomato application container image.
Amazon ECR	Stores the private Docker image.
Amazon EKS	Runs and manages the Kubernetes environment.
EKS Auto Mode	Provides and manages Kubernetes compute capacity.
Kubernetes	Manages Deployments, Pods and Services.
NodePort Service	Exposes the Zomato application through NodePort 31403.
Helm	Installs and manages the Prometheus Community monitoring stack.
Prometheus	Collects Kubernetes and workload metrics.
Grafana	Visualizes Prometheus metrics through dashboards.
Alertmanager	Provides alert management for the monitoring stack.
Node Exporter	Collects node-level metrics.
Kube State Metrics	Exposes Kubernetes object/state metrics.
Prometheus Operator	Manages Prometheus-related Kubernetes monitoring resources.
metrics-server	Provides Kubernetes resource metrics used by the cluster.

4. Implementation & Screenshots

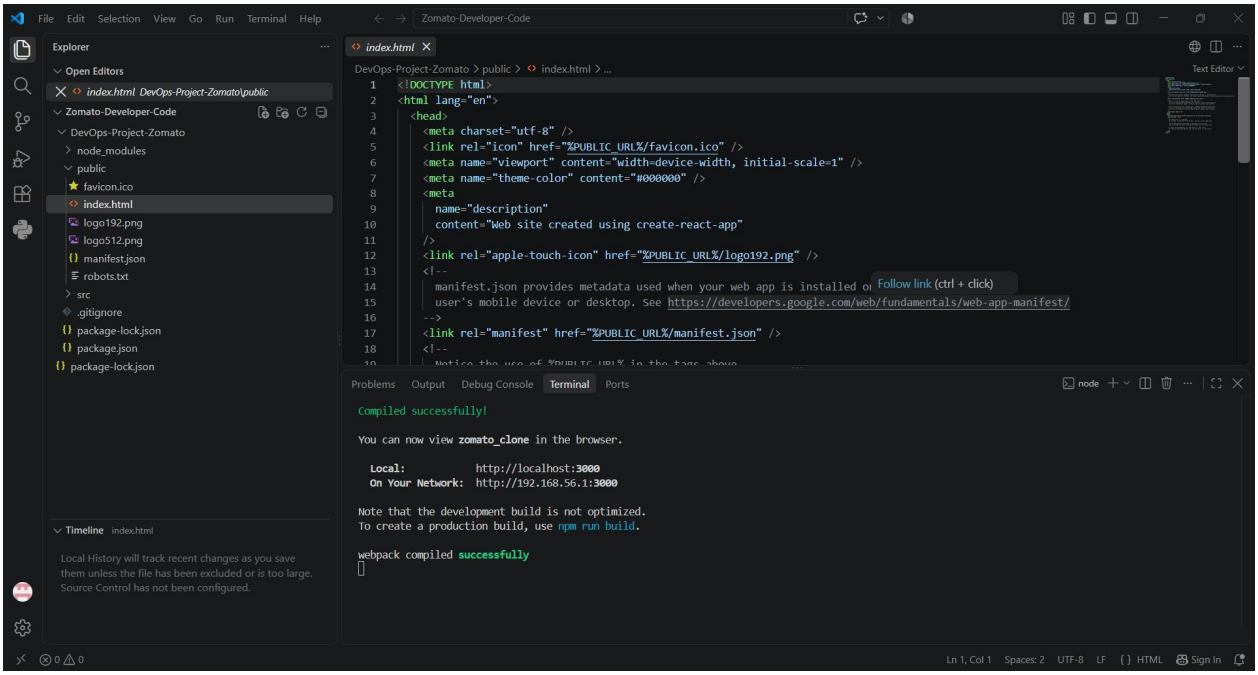
Application Source Control

SS 1 : Zomato Application Running in Local Development Environment



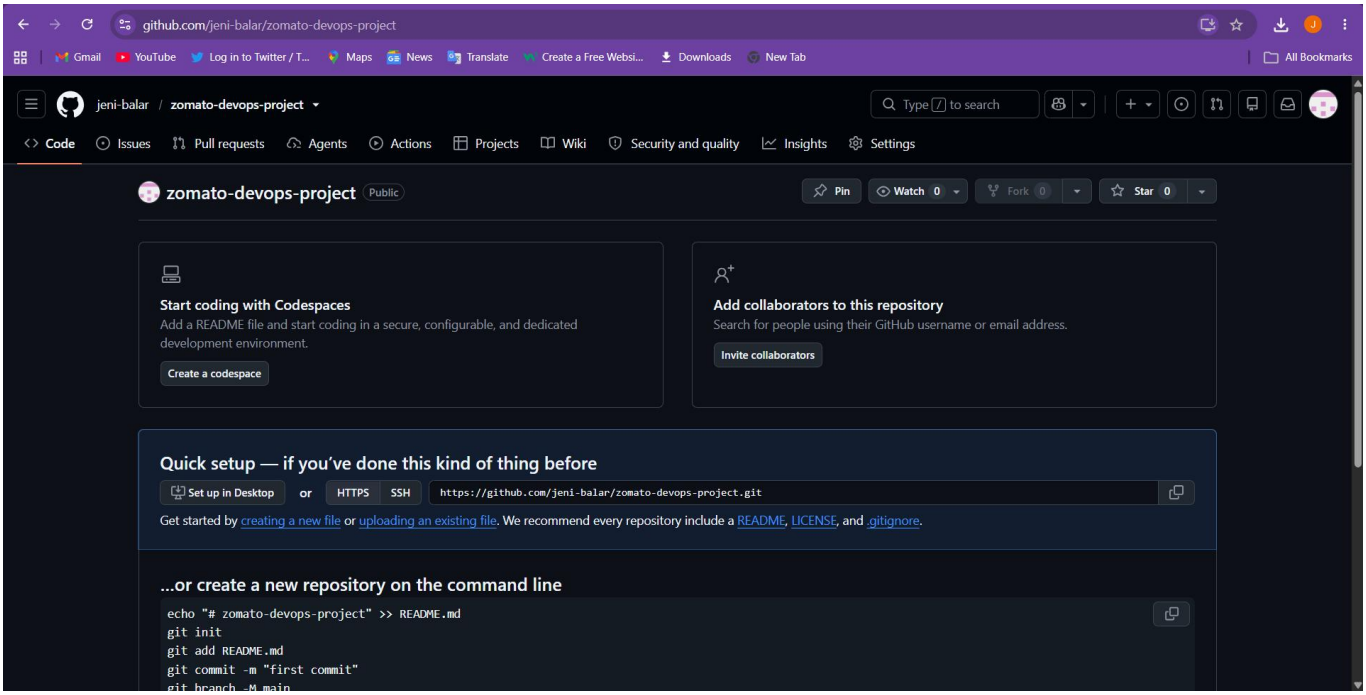
- The Zomato application was first verified in the local development environment. This confirmed that the application was functional and ready to be integrated into the DevOps workflow.

SS 2 : Zomato Project Source-Code Structure



- The project source-code structure was reviewed and used as the foundation for source control, containerization, CI/CD automation and Kubernetes deployment.

SS 3 : Empty GitHub Repository Created

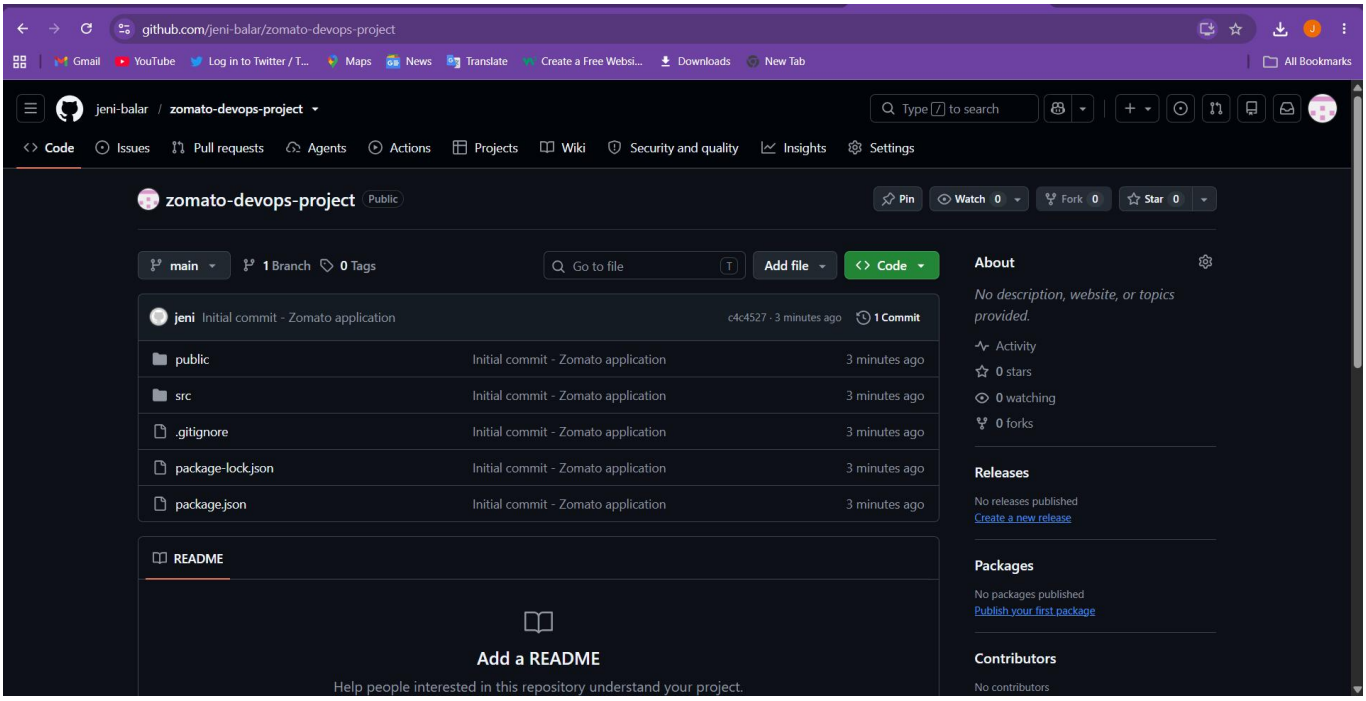


- An empty GitHub repository was created to host the Zomato application's source code and provide the source-control endpoint for the Jenkins CI/CD pipeline

SS 4 : Zomato Application Source Code Pushed to GitHub

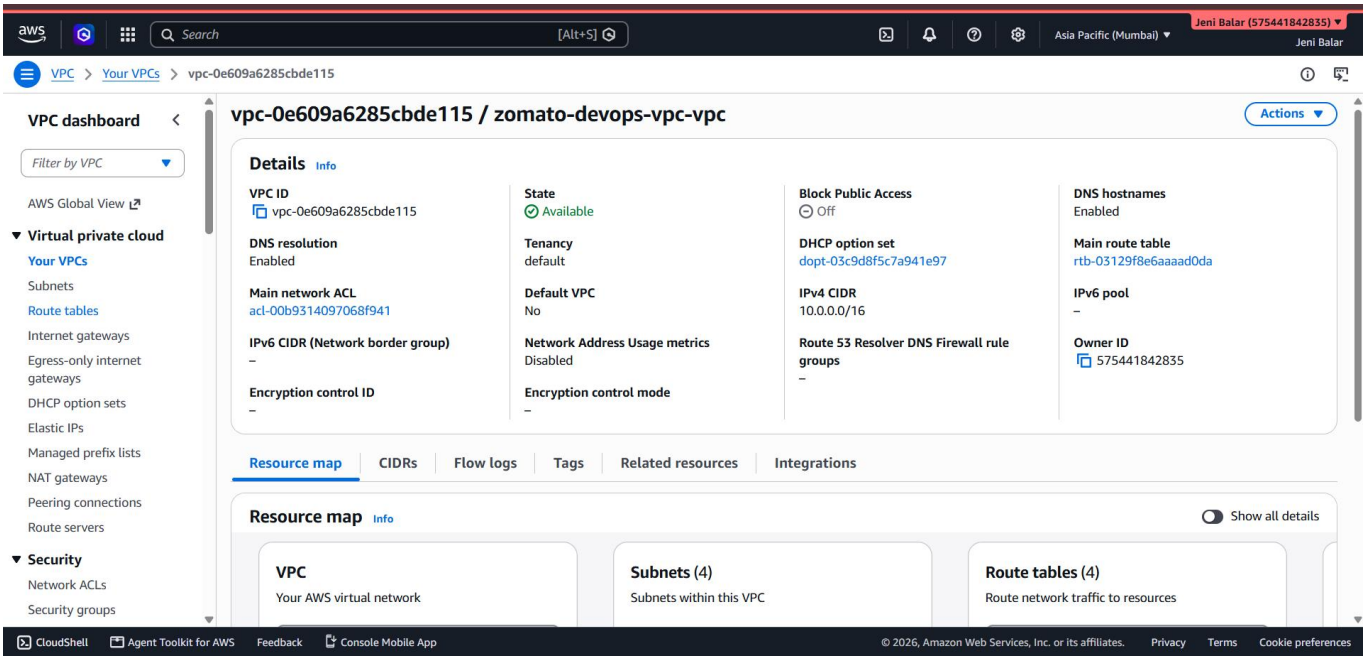
- The Zomato application source code was successfully pushed to the main branch of the GitHub repository, establishing GitHub as the source for the automated delivery workflow.

SS 5: Zomato Source Code Verified in GitHub



- The main GitHub branch was verified and confirmed to contain the Zomato application source code required for the subsequent Jenkins build and deployment stages.

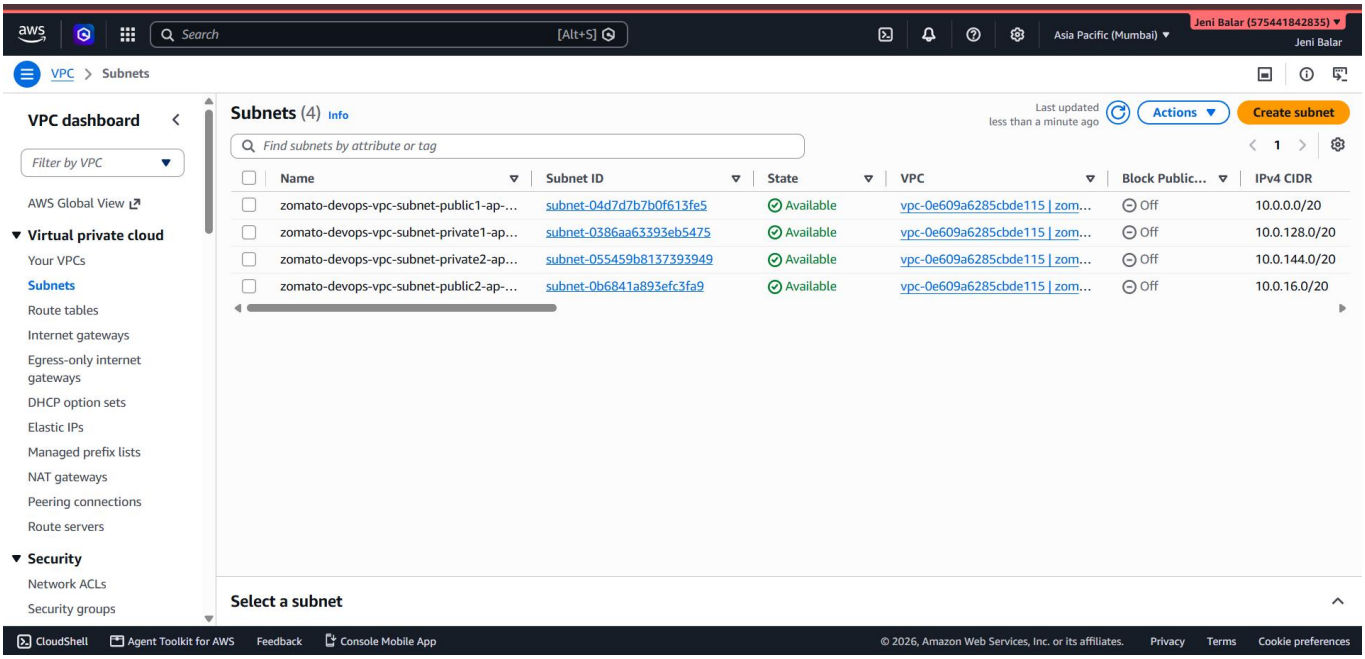
SS 6 : AWS VPC Created for Zomato DevOps Infrastructure



- A dedicated AWS VPC with CIDR 10.0.0.0/16 was created as the networking foundation for the Zomato DevOps infrastructure.

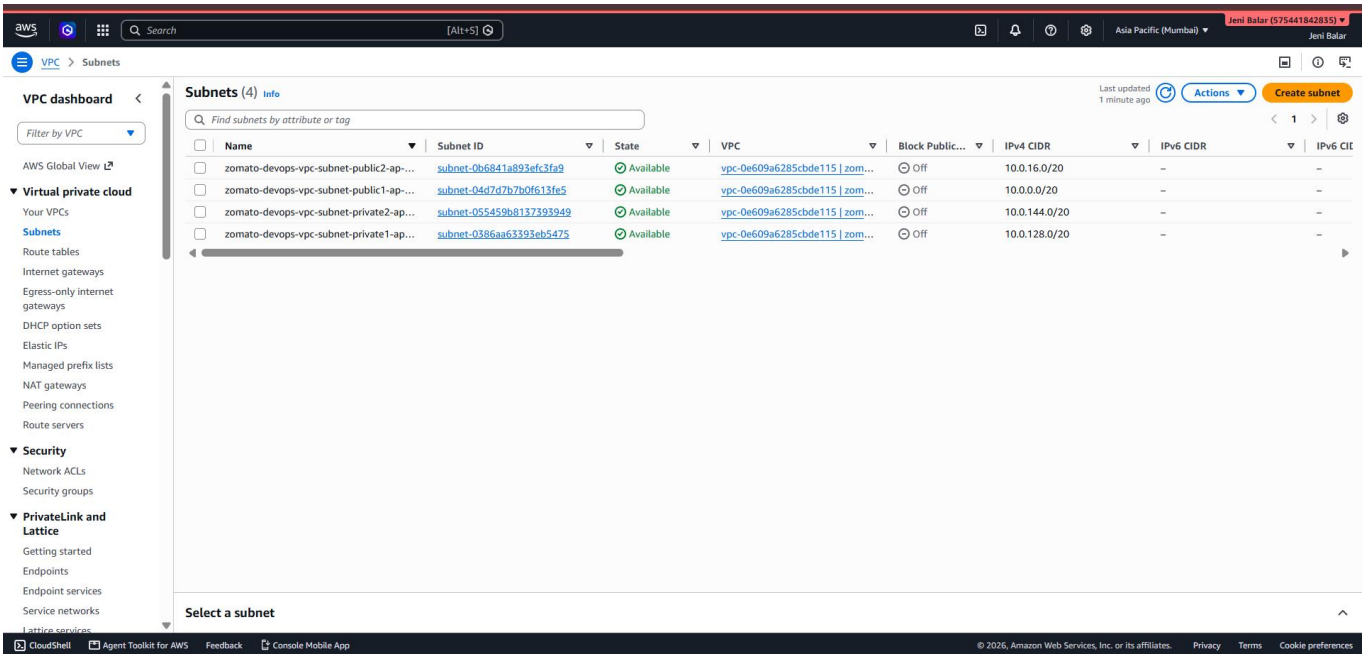
AWS Networking & Jenkins CI/CD Infrastructure

SS 7 : Public and Private Subnets Across Two Availability Zones



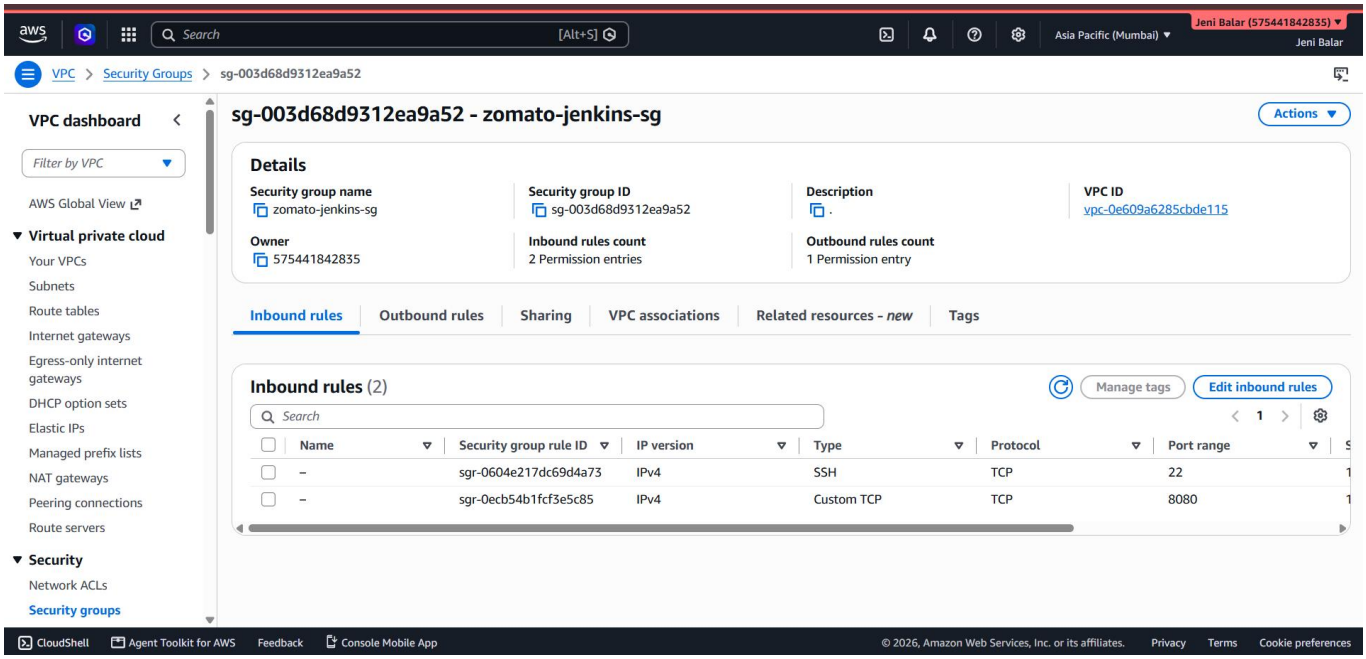
- Four subnets were created across two Availability Zones to separate public and private workloads and provide the network layout used by the project.

SS 8 : AWS VPC Resource Map



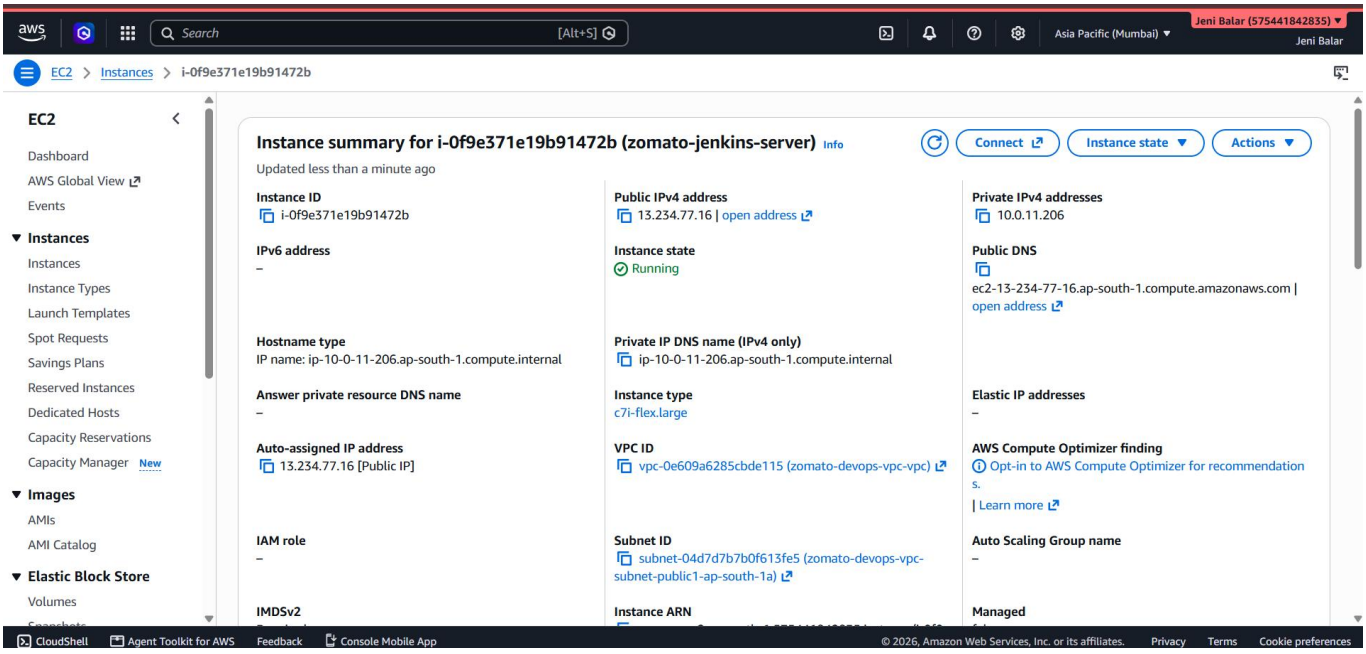
- The VPC resource map was verified, showing the Internet Gateway, public subnets, NAT Gateway and private subnets used by the Zomato DevOps environment.

SS 9 : Jenkins Security Group Configuration



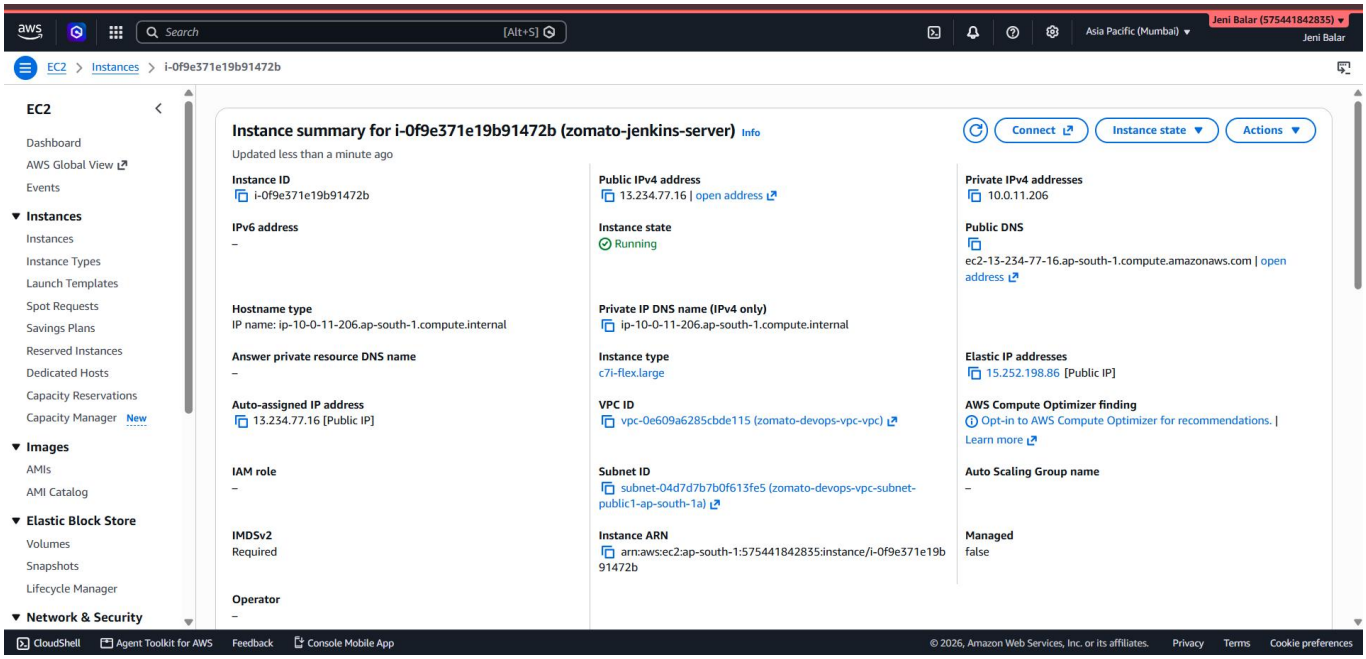
- The Jenkins security group was configured to provide the required network access for the EC2-based Jenkins CI/CD server.

SS 10 : Jenkins EC2 Server Successfully Launched



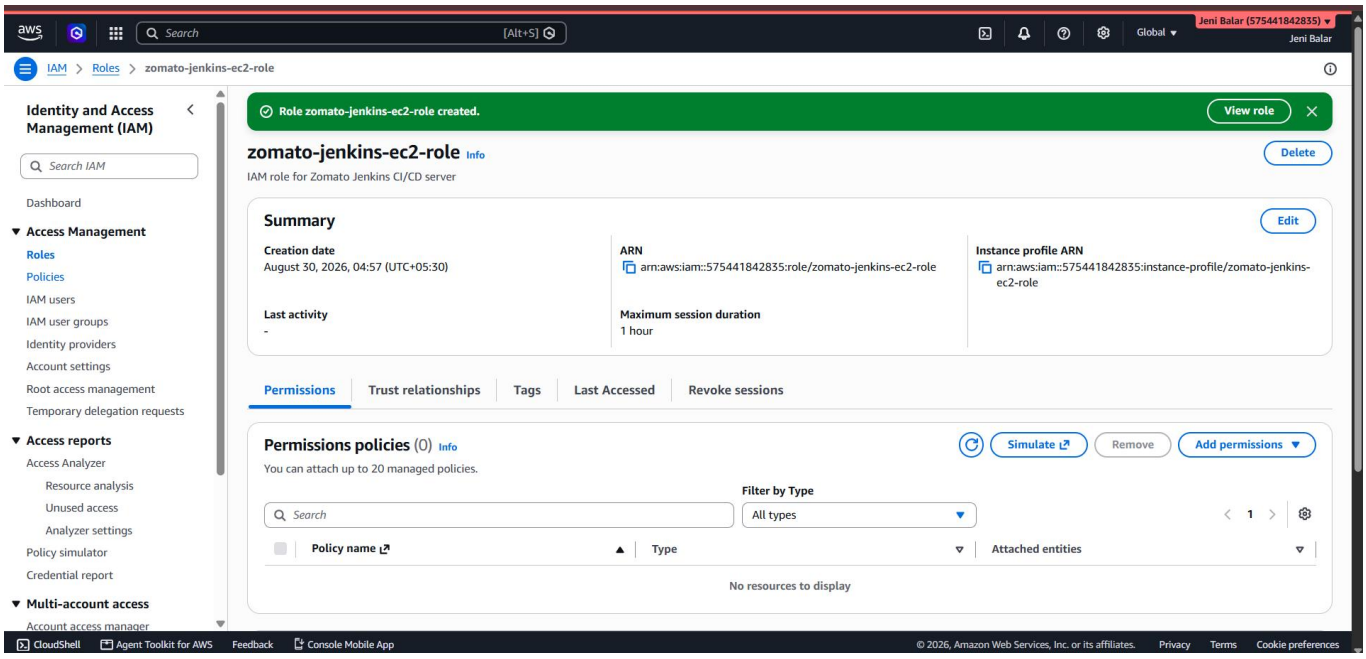
- The Jenkins EC2 server was launched in the Zomato DevOps VPC and configured to host the CI/CD environment.

SS 11 : Elastic IP Associated with Jenkins EC2 Server



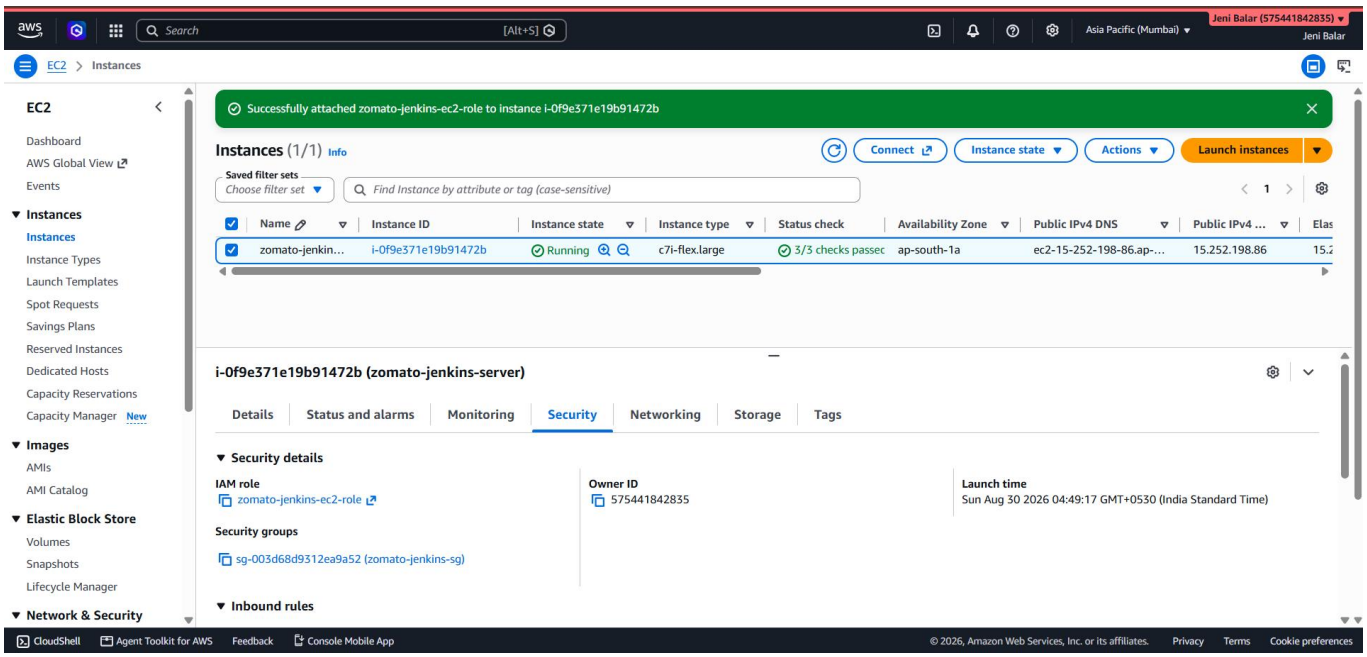
- An Elastic IP address was associated with the Jenkins EC2 server to provide a stable public endpoint for administrative and application access.

SS 12 : Jenkins IAM Role Configured



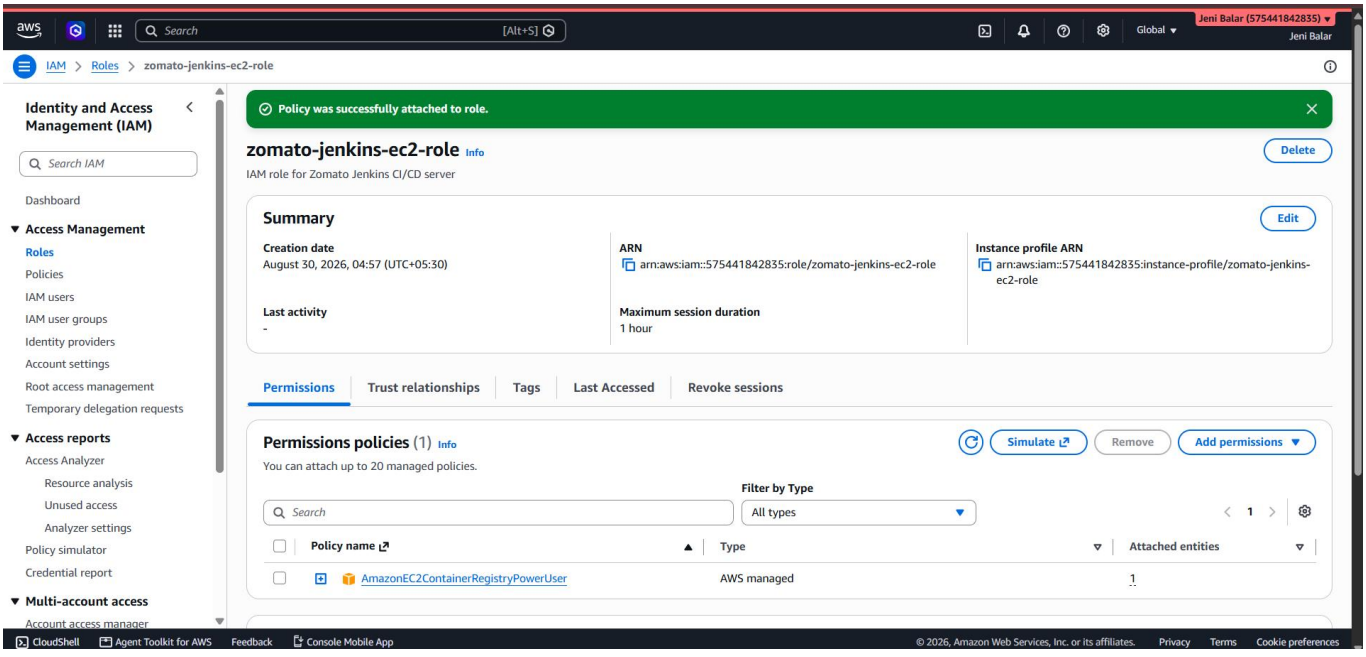
- An IAM role was configured for the Jenkins EC2 server so that AWS access could be provided through an attached role rather than storing long-lived access keys on the server.

SS 13 : Jenkins IAM Role Successfully Attached



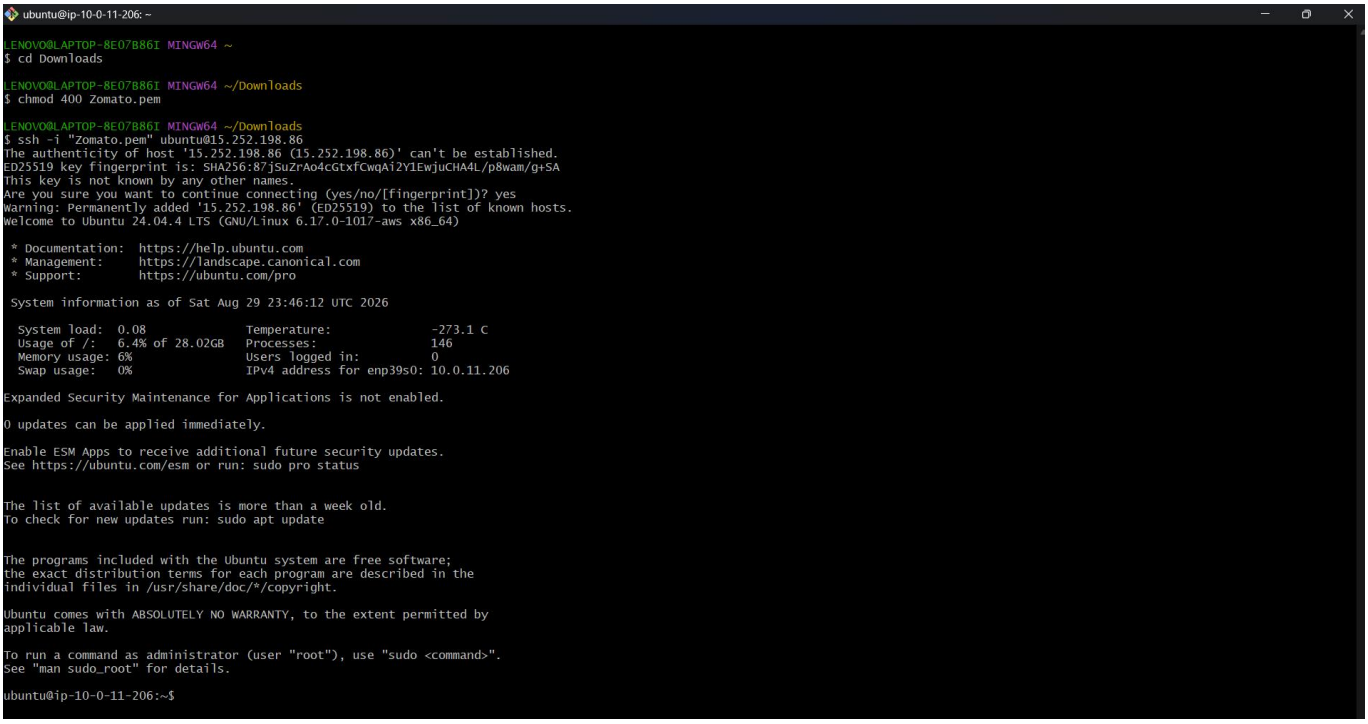
- The dedicated IAM role was successfully attached to the Jenkins EC2 instance, enabling the server to use the permissions required by the DevOps workflow.

SS 14 : Amazon ECR Permissions Attached to Jenkins Role



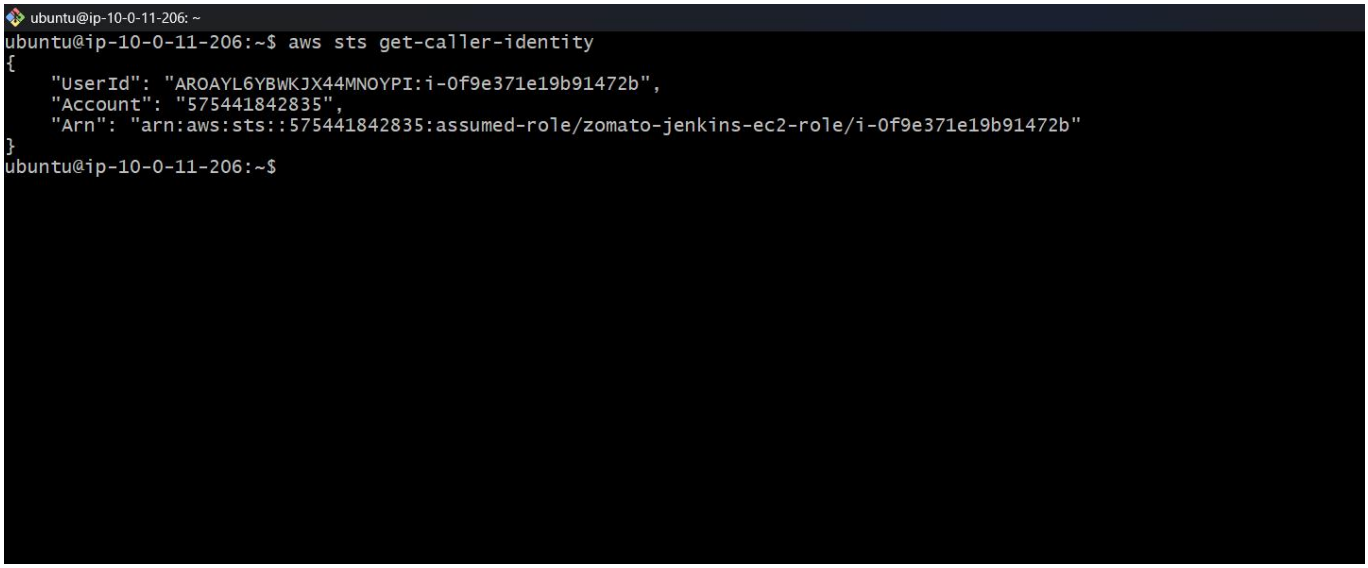
- Amazon ECR permissions were attached to the Jenkins IAM role so Jenkins could authenticate with and publish Docker images to the private ECR repository.

SS 15 : SSH Connection to Jenkins EC2 Server



- A successful SSH connection to the Jenkins EC2 server was established using Git Bash, confirming administrative access to the CI/CD host.

SS 16 : IAM Role Authentication Verification



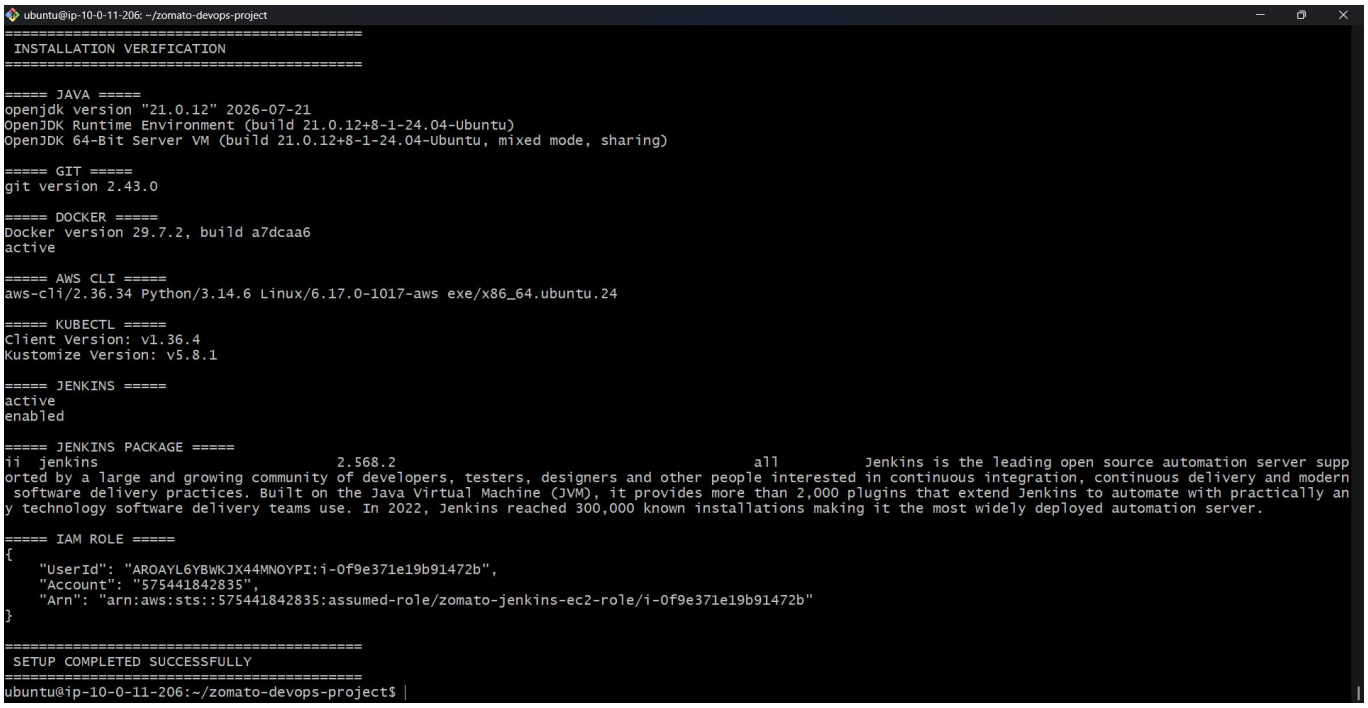
- AWS identity and role-based authentication were verified from the Jenkins environment, confirming that the EC2 instance could access AWS services through its IAM role.

SS 17 : Application Security Group Rule

```
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git push origin main
Username for 'https://github.com': jeni-balar
Password for 'https://jeni-balar@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.56 KiB | 1.56 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/jeni-balar/zomato-devops-project.git
   c4c4527..07248a1  main -> main
ubuntu@ip-10-0-11-206:~/zomato-devops-project$
```

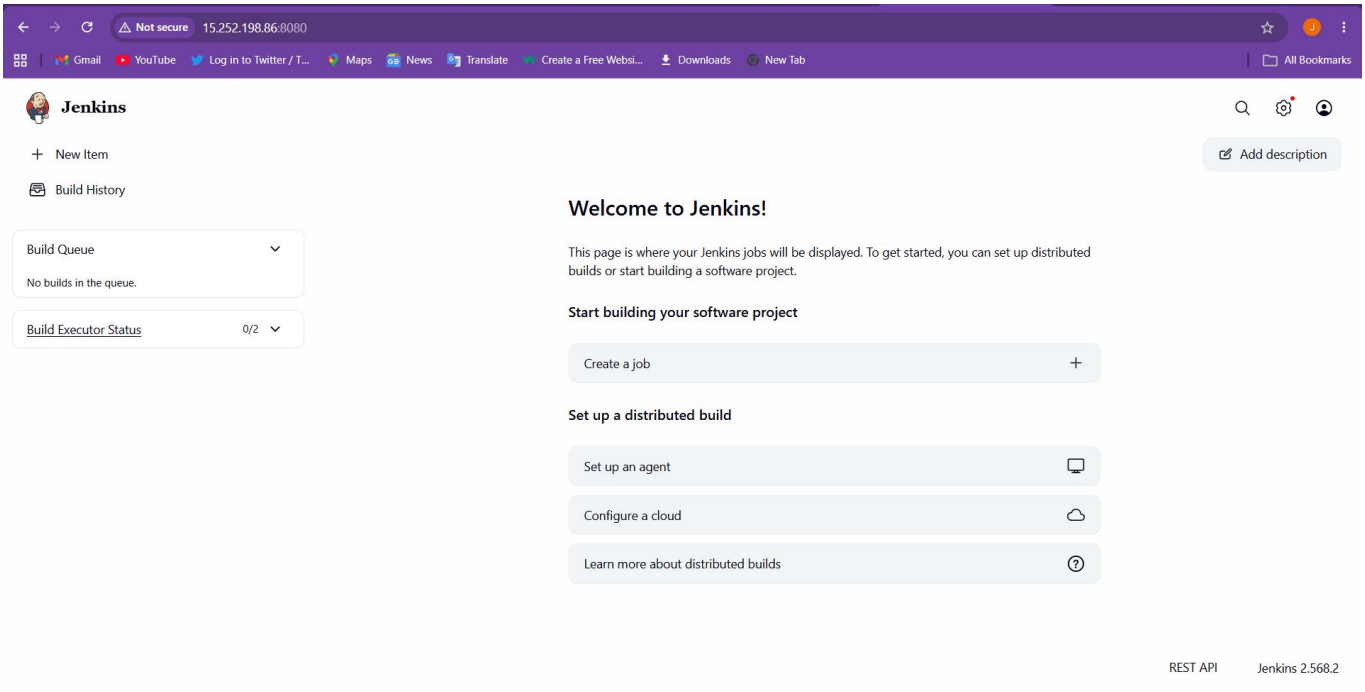
- The Jenkins server setup script was added to the project and pushed to the main GitHub branch, keeping the environment setup steps version-controlled with the project.

SS 18 : Jenkins Environment Setup Completed



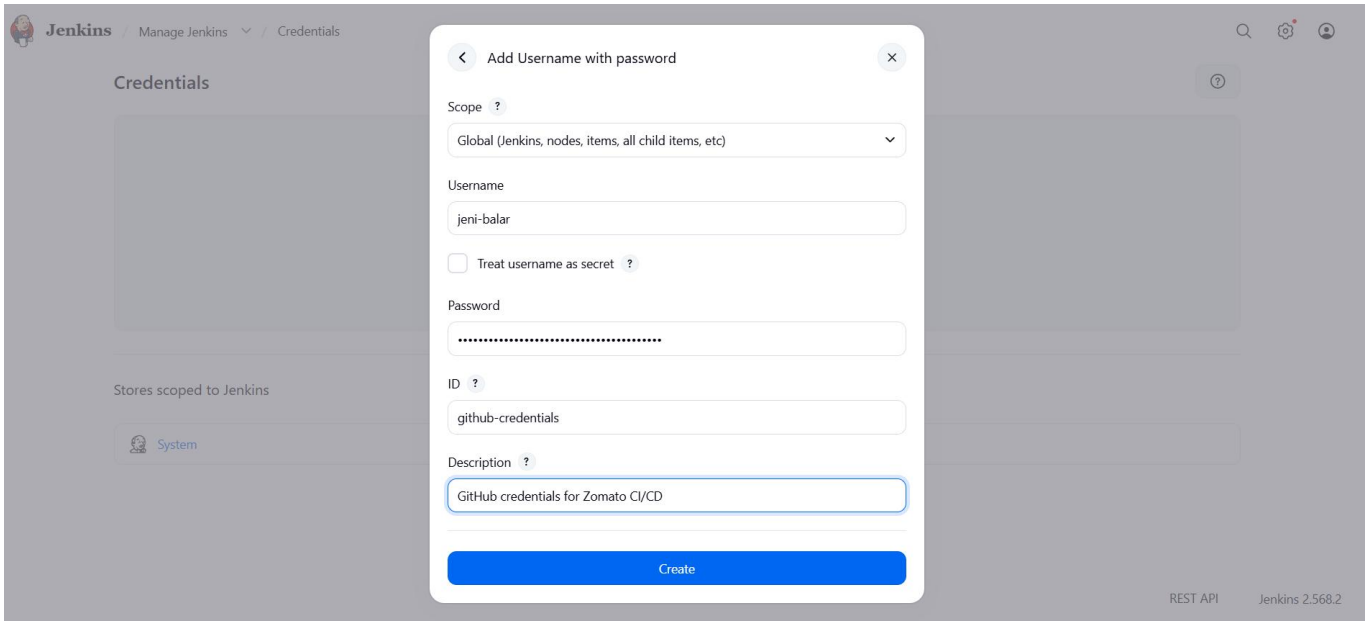
- The automated Jenkins setup and verification completed successfully, including the required Java, Git, Docker, AWS CLI, kubectl, Jenkins and supporting DevOps tooling.

SS 19 : Jenkins Dashboard Successfully Accessed



- Jenkins was unlocked and configured through its initial setup, and the Jenkins dashboard was successfully accessed, confirming that the CI/CD server was ready for pipeline configuration

SS 20 : GitHub Credentials Added to Jenkins



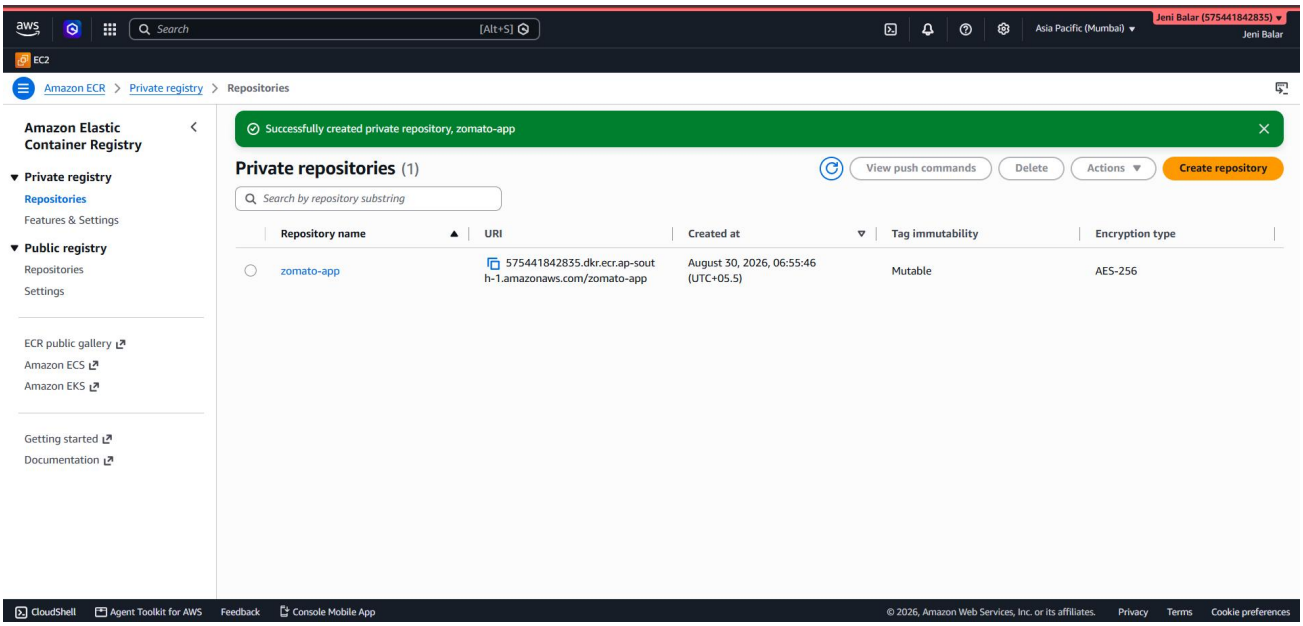
- GitHub credentials were securely added to Jenkins so the pipeline could authenticate with the repository and retrieve the Zomato project source code.

SS 21 : Dockerfile, Jenkinsfile and Setup Files Pushed

```
node_modules
build
.git
.gitignore
Dockerfile
Jenkinsfile
README.md
npm-debug.log
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        Dockerfile
        Jenkinsfile

nothing added to commit but untracked files present (use "git add" to track)
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git add Dockerfile .dockerignore Jenkinsfile scripts/setup-jenkins.sh
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git commit -m "Add Docker and Jenkins CI/CD configuration"
[main 05c644d] Add Docker and Jenkins CI/CD configuration
 Committer: Ubuntu <ubuntu@ip-10-0-11-206.ap-south-1.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 3 files changed, 171 insertions(+)
 create mode 100644 .dockerignore
 create mode 100644 Dockerfile
 create mode 100644 Jenkinsfile
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git push origin main
Username for 'https://github.com': jeni-balar
Password for 'https://jeni-balar@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.66 KiB | 1.66 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/jeni-balar/zomato-devops-project.git
   c92d225..05c644d  main -> main
ubuntu@ip-10-0-11-206:~/zomato-devops-project$
```

- The Dockerfile, Docker ignore rules, Jenkinsfile and Jenkins setup script were added to the project and pushed to the main GitHub branch for use by the CI/CD workflow.

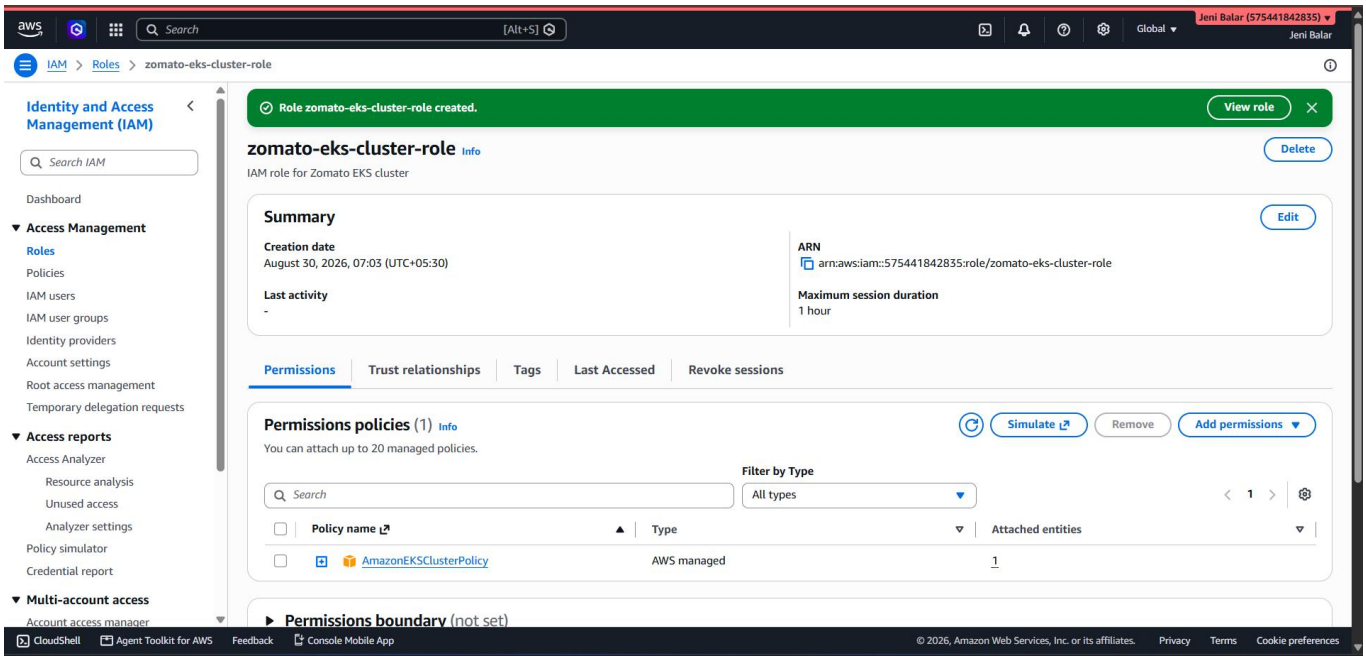
SS 22 : Amazon ECR Repository Created



- The zomato-app Amazon ECR repository was created successfully and prepared to store Docker images generated by the Jenkins pipeline.

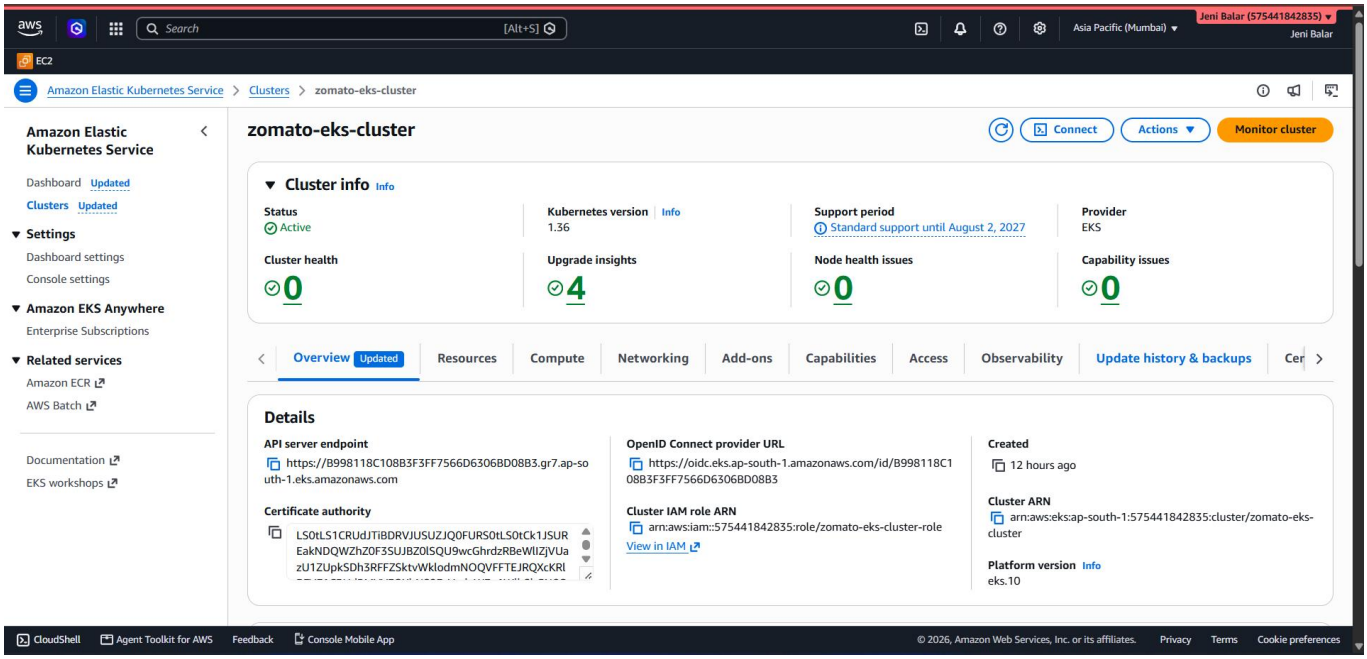
Amazon ECR, EKS & Kubernetes Deployment

SS 23 : EKS Cluster IAM Role Created



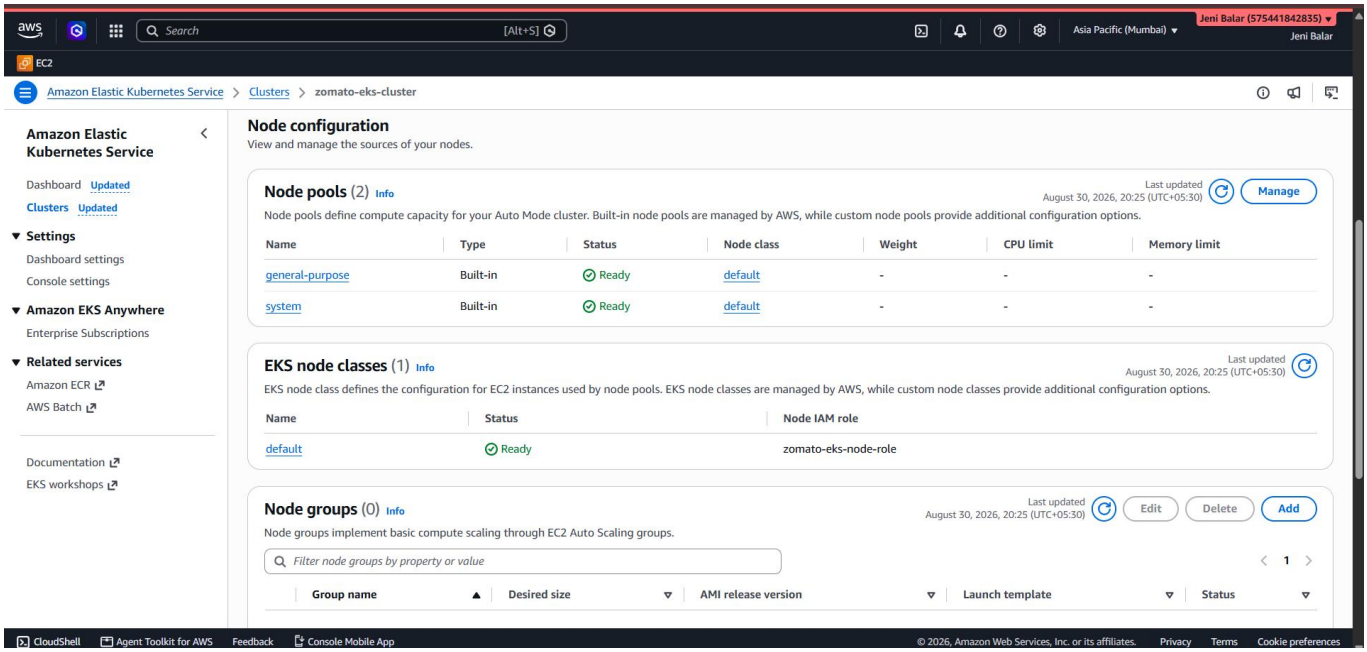
- The IAM role required by the Amazon EKS cluster was created with the required EKS cluster permissions.

SS 24 : Amazon EKS Cluster Created Successfully



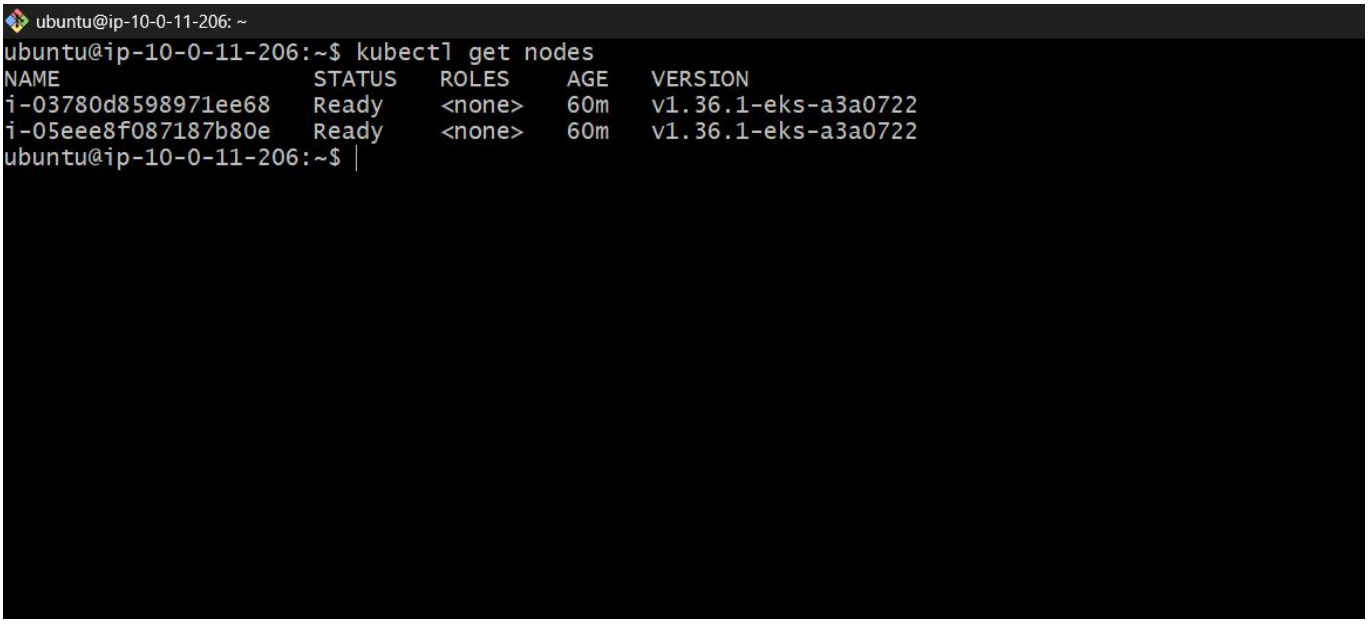
- The Amazon EKS cluster was created successfully and reached the Active state using Kubernetes 1.36.

SS 25 : EKS Auto Mode Node Pools Ready



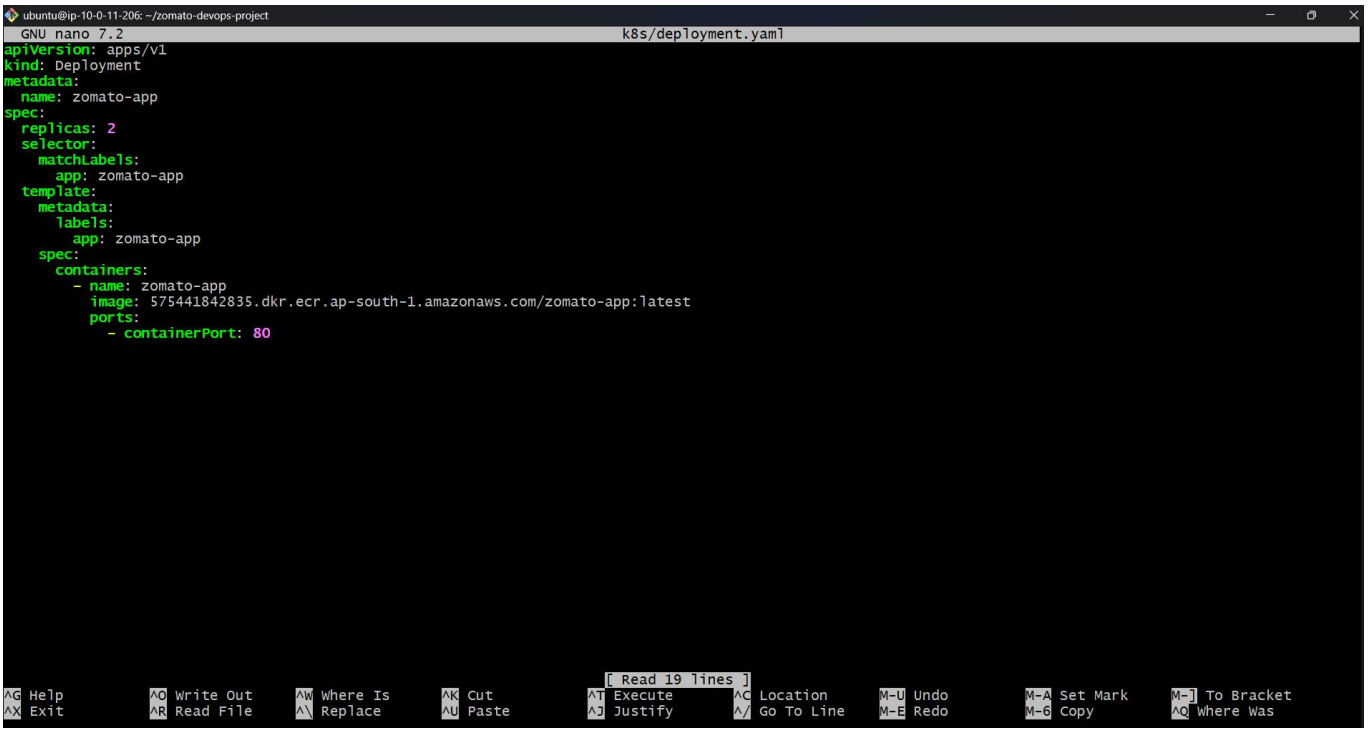
- EKS Auto Mode was verified with the general-purpose and system node pools ready, providing the compute capacity required for Kubernetes workloads.

SS 26 : EKS Cluster Nodes in Ready State



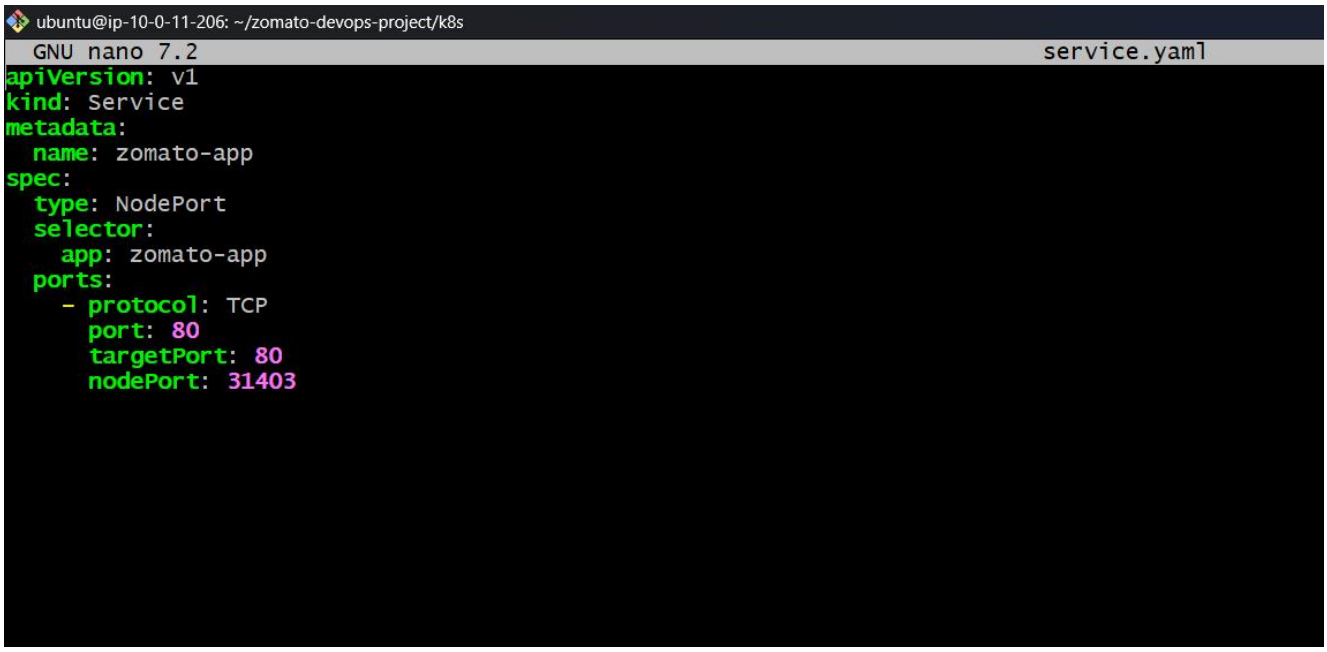
- The EKS cluster nodes were verified with kubectl and shown in the Ready state, confirming that the Kubernetes compute environment was operational.

SS 27 : Kubernetes Deployment Manifest



- The Kubernetes Deployment manifest was configured with two application replicas and the Zomato container image stored in Amazon ECR.

SS 28 : Kubernetes Service Configuration – NodePort 31403



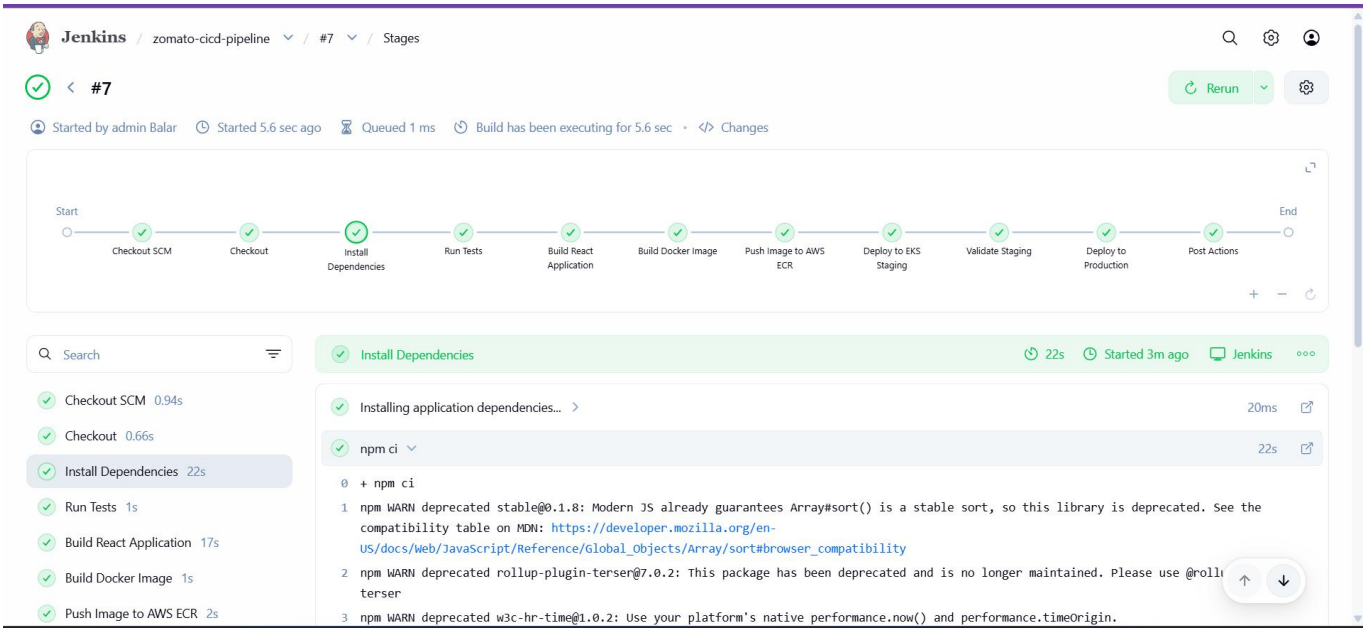
- The Kubernetes Service was configured as a NodePort service, exposing the Zomato application on service port 80 through NodePort 31403.

SS 29 : Kubernetes Configuration Pushed to GitHub

```
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Jenkinsfile
        new file:   k8s/deployment.yaml
        new file:   k8s/service.yaml

ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git commit -m "Add Kubernetes deployment configuration"
[main 6e9de51] Add Kubernetes deployment configuration
 Committer: Ubuntu <ubuntu@ip-10-0-11-206.ap-south-1.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 3 files changed, 32 insertions(+), 1 deletion(-)
 create mode 100644 k8s/deployment.yaml
 create mode 100644 k8s/service.yaml
ubuntu@ip-10-0-11-206:~/zomato-devops-project$ git push origin main
Username for 'https://github.com': jeni-balar
Password for 'https://jeni-balar@github.com':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 798 bytes | 798.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/jeni-balar/zomato-devops-project.git
   05c644d..6e9de51  main -> main
ubuntu@ip-10-0-11-206:~/zomato-devops-project$
```

- The Kubernetes deployment and service configuration changes were committed and pushed to the main GitHub branch, keeping the deployment manifests version-controlled.

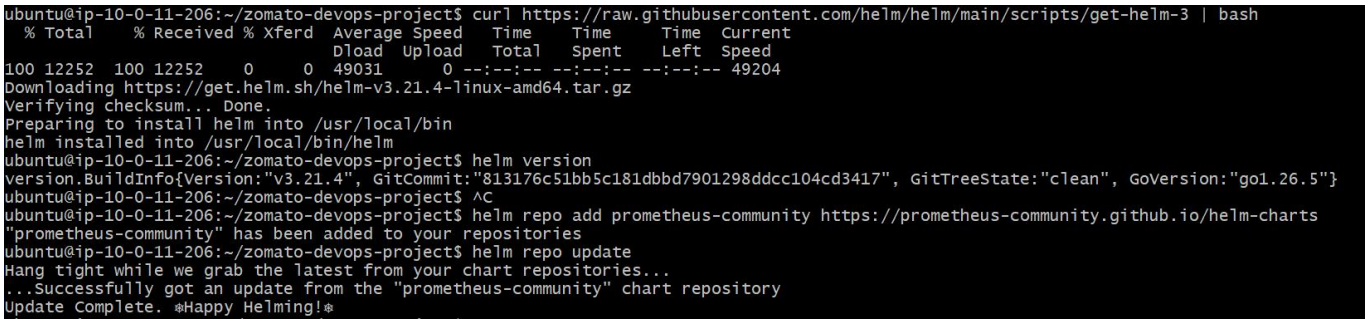
SS 30 : Connecting to RDS and Creating Database



- The Jenkins CI/CD pipeline completed successfully, confirming the automated build, Docker image creation and push to Amazon ECR, followed by Kubernetes deployment to Amazon EKS

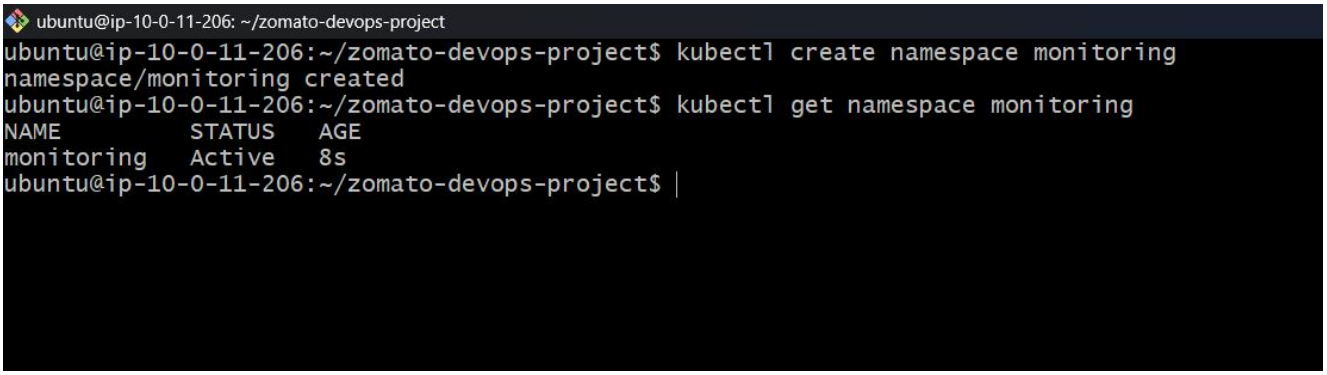
Monitoring, Grafana & Final Verification

SS 31 : Helm and Prometheus Community Repository Configuration



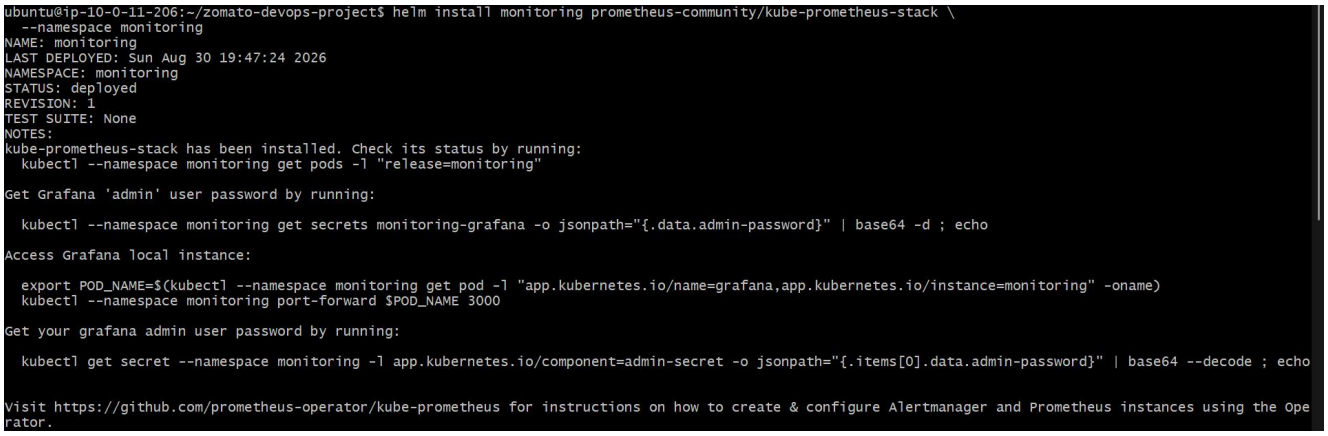
- Helm was installed and the Prometheus Community repository was configured to prepare the Kubernetes environment for the monitoring stack.

SS 32 : Products Table Creation and Verification



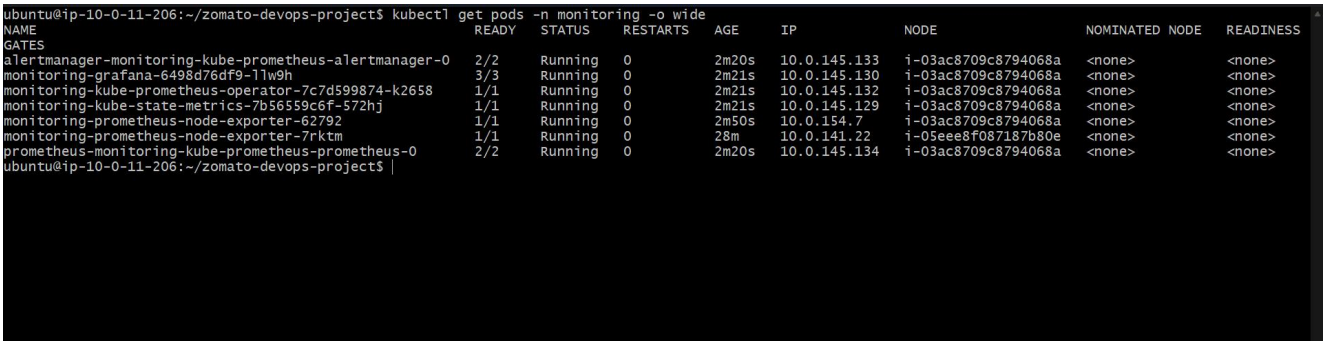
- A dedicated monitoring namespace was created in the EKS cluster to isolate and organize the Prometheus and Grafana monitoring components.

SS 33 : Prometheus and Grafana Monitoring Stack Installed



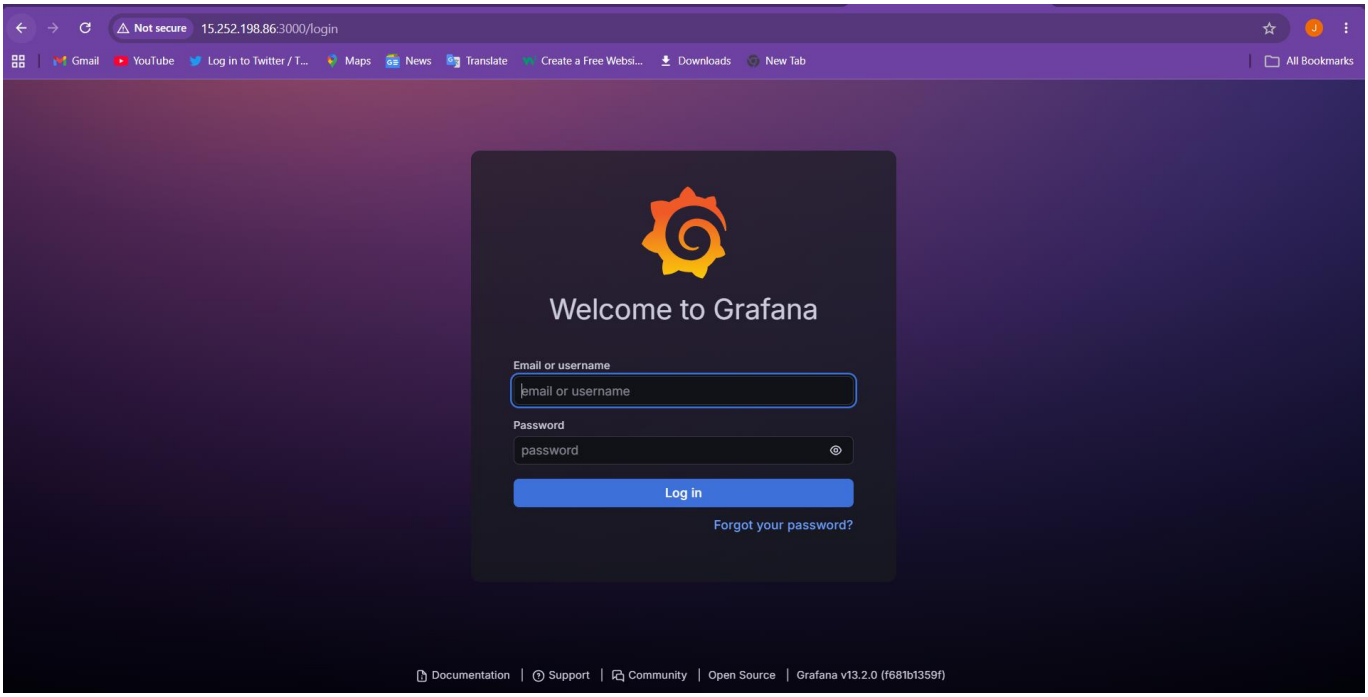
- The Prometheus Community monitoring stack was successfully deployed in the monitoring namespace using Helm.

SS 34 : Monitoring Stack Pods Healthy



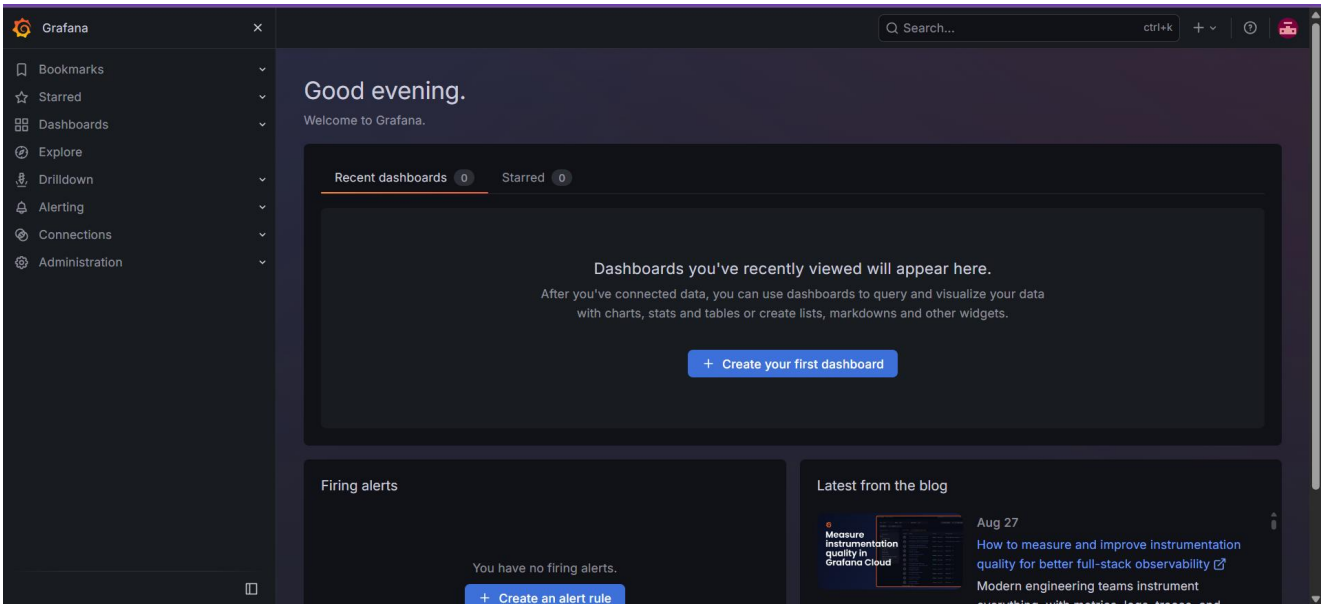
- Prometheus, Grafana, Alertmanager, Prometheus Operator, Kube State Metrics and Node Exporter components were verified as running in the monitoring namespace.

SS 35 : Grafana Successfully Deployed



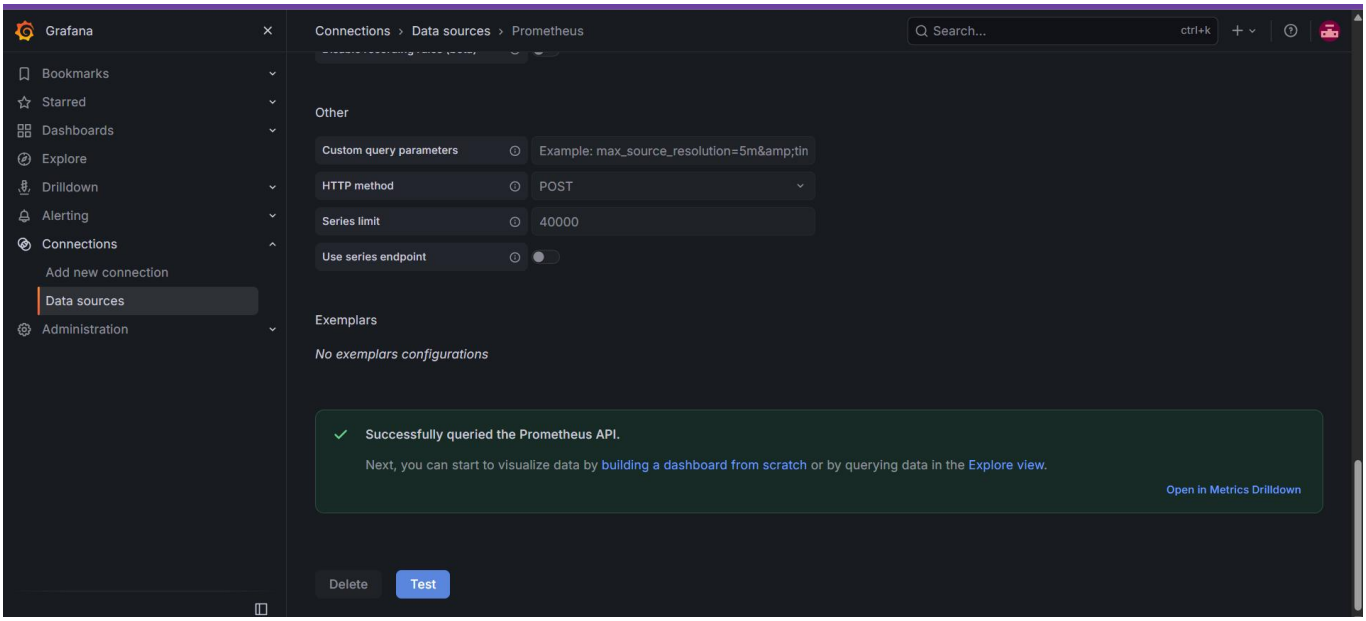
- Grafana was successfully deployed on the EKS cluster and accessed through Kubernetes port forwarding for monitoring configuration and dashboard creation.

SS 36 : Grafana Dashboard Access



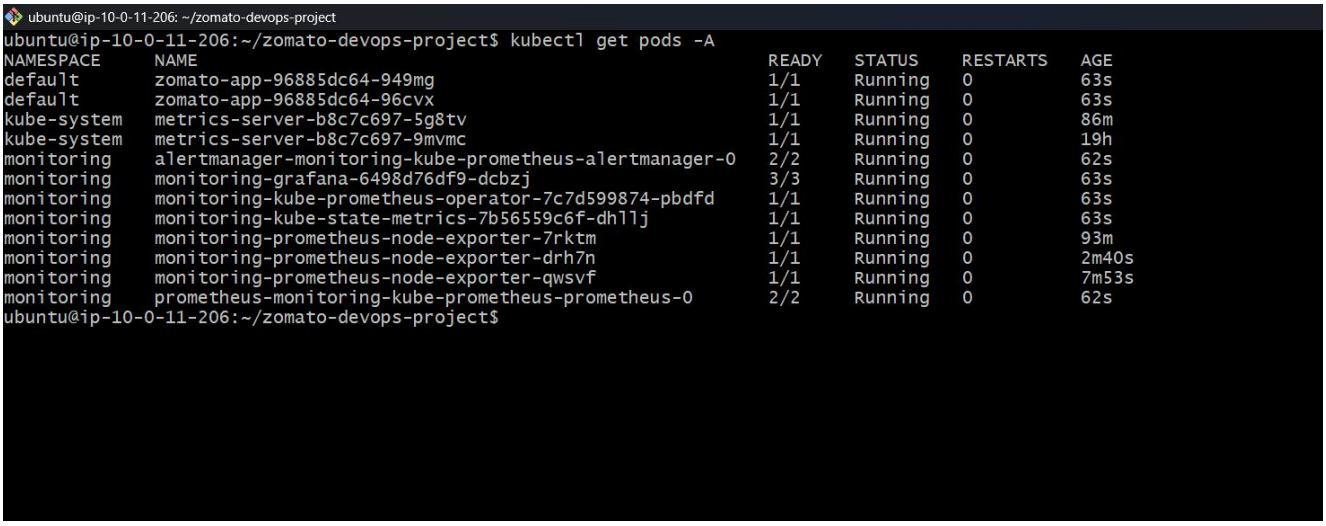
- The Grafana web interface was successfully accessed and prepared for visualizing Kubernetes metrics collected through Prometheus.

SS 37 : Prometheus Data Source Connected to Grafana



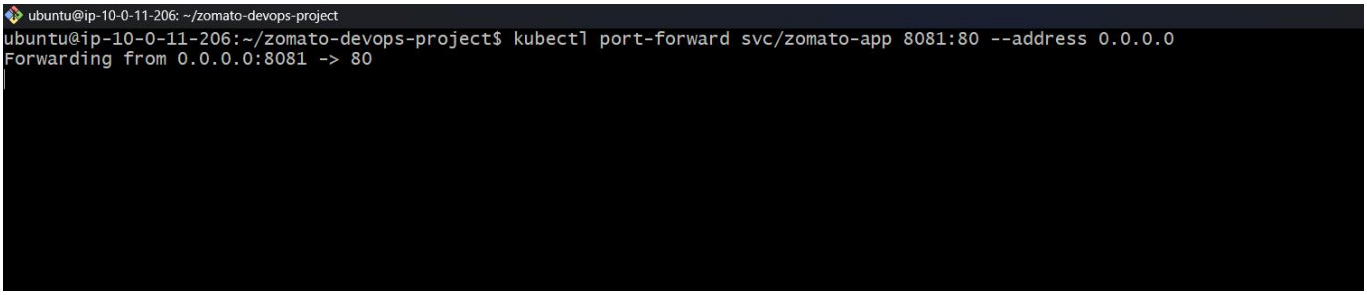
- Prometheus was configured as the Grafana data source and the connection was verified successfully, making the collected metrics available for dashboard visualization.

SS 38 : Final Kubernetes Pod Status



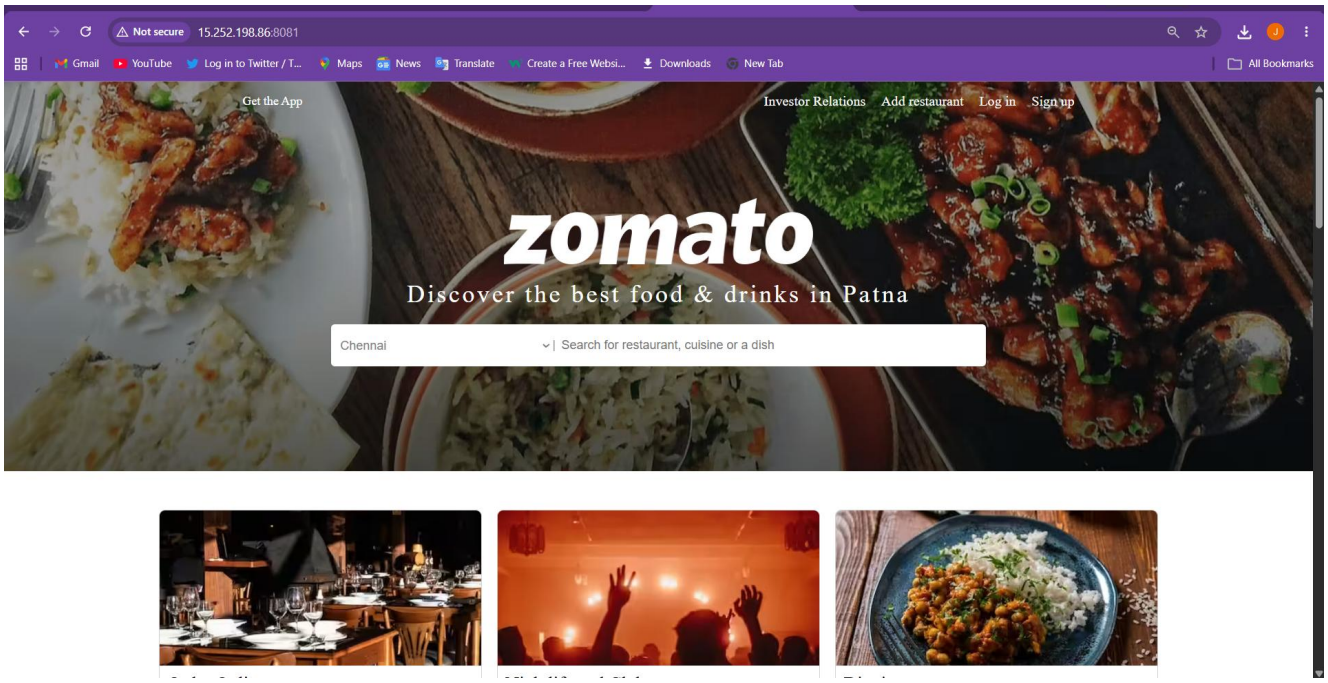
- The final Kubernetes status check confirmed that the Zomato application and monitoring components were running successfully with zero restarts.

SS 39 : Zomato Application Port Forwarding



- The zomato-app Kubernetes service was successfully port-forwarded from service port 80 to port 8081 on the Jenkins EC2 host, enabling browser-based application verification.

SS 40 : Zomato Application Successfully Deployed on Amazon EKS



- The Zomato web application was successfully accessed through the EC2 port-forwarding path, confirming container deployment, Kubernetes service exposure and end-to-end application connectivity.

5. Kubernetes and Deployment Design

- The Zomato application is packaged as a Docker image and deployed to Amazon EKS. The Kubernetes Deployment defines the application workload, while the Kubernetes Service of type NodePort exposes service port 80 through NodePort 31403. The deployment manifests are maintained in the project repository and applied to the EKS cluster.

5.1 Deployment Sequence

- Jenkins obtains the latest source code and builds the Docker image.
- The image is pushed to the private zomato-app Amazon ECR repository.
- Kubernetes uses the ECR image in the application Deployment.
- The Deployment maintains two application replicas on the EKS worker infrastructure.
- The zomato-app Service routes traffic to the application Pods and exposes NodePort 31403.
- Final verification uses kubectl port-forward svc/zomato-app 8081:80 --address 0.0.0.0 to provide browser access through the Jenkins EC2 host.

5.2 Kubernetes Resources

- The implementation uses a Kubernetes Deployment for the Zomato application, a NodePort Service for application exposure, application Pods created by the Deployment, and a dedicated monitoring namespace containing the Prometheus Community monitoring components.

6. Monitoring and Observability

- The project implements Kubernetes monitoring using the Prometheus Community monitoring stack installed through Helm. Prometheus collects cluster and workload metrics, while Grafana uses Prometheus as its data source to visualize the collected information.

    - Prometheus monitoring and metrics collection verified.
    - Grafana deployed and accessed successfully.
    - Prometheus configured as the Grafana data source.
    - Alertmanager deployed as part of the monitoring stack.
    - Node Exporter deployed for node-level metrics.
    - Kube State Metrics deployed for Kubernetes object/state metrics.
    - Prometheus Operator deployed to manage Prometheus monitoring resources.
    - metrics-server running in the Kubernetes system namespace.
    - Grafana dashboard completed with six panels: CPU Usage, Memory Usage, Node Count, Pod Count, Pod CPU Usage and Pod Memory Usage.

7. Key Verification Commands

- kubectl get pods -A
- kubectl get svc
- kubectl get nodes -o wide
- kubectl get deployment
- kubectl get endpoints -n monitoring monitoring-grafana
- kubectl get svc -n monitoring monitoring-grafana
- kubectl get pods -n monitoring
- kubectl get nodes
- kubectl rollout status deployment/zomato-app
- kubectl get all
- kubectl port-forward svc/zomato-app 8081:80 --address 0.0.0.0
- aws sts get-caller-identity
- aws ecr describe-repositories --repository-names zomato-app --region ap-south-1.

8. Troubleshooting

1. Grafana Connectivity Issue

Problem: Grafana was deployed successfully, but it was not directly accessible from the browser because the Grafana Kubernetes Service was configured as ClusterIP.
Cause: A ClusterIP service is accessible only within the Kubernetes cluster.
Solution: The Grafana service and its endpoint were verified, and Kubernetes port forwarding was used to access Grafana for dashboard configuration and monitoring.

2. Kubernetes Application Port Conflict

Problem: The initial port-forwarding command for the Zomato application failed because port 8080 was already being used on the EC2 server.
Cause: Another service was already listening on port 8080.
Solution: Instead of stopping the existing service, an unused port 8081 was selected:
kubectl port-forward svc/zomato-app 8081:80 --address 0.0.0.0
The application was then successfully accessed through port 8081.

3. Grafana Pod Readiness Issue

Problem: After deploying the monitoring stack, Grafana initially showed a readiness probe failure with connection refused.
Cause: Grafana was still starting up and the application was not yet listening on port 3000 when Kubernetes performed the readiness check.
Solution: The Grafana Pod status and events were monitored until the container became ready. Once Grafana reached the Running/Ready state, the dashboard was accessed successfully.

9. End-to-End Verification

- Zomato source code is stored in GitHub.
- Jenkins successfully performs the CI/CD build and deployment workflow.
- Docker image is successfully stored in the private Amazon ECR repository.
- Amazon EKS cluster and Kubernetes nodes are operational.
- Zomato application Pods are running with the configured replicas.
- The Kubernetes zomato-app Service is configured as NodePort 31403.
- Prometheus and Grafana monitoring components are running in the monitoring namespace.
- Prometheus is configured as the Grafana data source.
- The Grafana dashboard displays six monitoring panels.
- The Zomato application is successfully accessed through the EC2 port-forwarding path.

10. Conclusion

The Zomato DevOps project successfully demonstrates an end-to-end DevOps workflow for deploying a web application on AWS. The implementation integrates GitHub, Jenkins, Docker, Amazon ECR, Amazon EKS, Kubernetes, Helm, Prometheus and Grafana to automate source management, containerization, image storage, Kubernetes deployment and monitoring.

The project establishes a dedicated AWS VPC with public and private subnets across two Availability Zones, uses an EC2-based Jenkins CI/CD server with IAM-based AWS access, deploys the application on Amazon EKS, and exposes it through a Kubernetes NodePort Service. Final browser verification is performed through port forwarding from the Jenkins EC2 host.

The monitoring implementation provides centralized visibility into the Kubernetes environment through Prometheus and Grafana, supported by Alertmanager, Node Exporter, Kube State Metrics, Prometheus Operator and metrics-server. The completed six-panel Grafana dashboard provides a concise operational view of CPU, memory, nodes and Pods.

Overall, the project demonstrates practical knowledge of AWS cloud infrastructure, CI/CD automation, Docker containerization, Amazon ECR, Kubernetes deployment, EKS, Helm-based monitoring and end-to-end application verification.